

Retrospective Evaluation of Soccer Players (REvSoc)

Seshadhri S

2025-07-18

2024-25 season

Goalkeeper Rating System: Methodology & Validation

In this analysis, I developed a multidimensional goalkeeper rating model designed to evaluate both conventional and modern aspects of goalkeeping performance. The model incorporates traditional shot-stopping metrics alongside advanced measures of distribution, sweeping actions, and cross management to reflect the evolving demands of the position.

Method Overview

The rating was divided into two core components:

Conventional Keeper Rating (CKR) — incorporating metrics like Saves, Goals Against (GA), Save Percentage, and Post-Shot Expected Goals (PSxG) overperformance.

Sweeper Keeper Rating (SKR) — capturing distribution accuracy, launch rates, defensive actions outside the box, average distance of defensive actions, and cross-stopping percentages.

Both components were normalized relative to their averages and combined to produce a Total Goalkeeper Rating.

Validation Process

To ensure the model was both credible and informative, I validated the Total Goalkeeper Rating against three independent metrics:

1. Goals Against (GA)
2. PSxG-GA per 90 (shot-stopping over expected)
3. Save Percentage

```
library(worldfootballR)
library(purrr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(Boruta)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats   1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v readr      2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(FactoMineR)
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(ggrepel)
```

Load in FBRef Data for the Big 5 Leagues

```
big5_player_standard <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "standard", team_or_player= "player")
big5_player_shooting <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "shooting", team_or_player= "player")
big5_player_passing <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "passing", team_or_player= "player")
big5_player_passing_types <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "passing_types", team_or_player= "player")
big5_player_gca <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "gca", team_or_player= "player")
big5_player_defense <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "defense", team_or_player= "player")
big5_player_possession <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "possession", team_or_player= "player")
big5_player_playing_time <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "playing_time", team_or_player= "player")
big5_player_misc <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "misc", team_or_player= "player")
big5_player_keepers <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "keepers", team_or_player= "player")
big5_player_keepers_adv <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "keepers_adv", team_or_player= "player")

prem_league_table <- fb_season_team_stats(country = "ENG", gender = "M", season_end_year = "2025", tier = "1")
seriea_league_table <- fb_season_team_stats(country = "ITA", gender = "M", season_end_year = "2025", tier = "1")
ligue1_league_table <- fb_season_team_stats(country = "FRA", gender = "M", season_end_year = "2025", tier = "1")
laliga_league_table <- fb_season_team_stats(country = "ESP", gender = "M", season_end_year = "2025", tier = "1")
bundesliga_league_table <- fb_season_team_stats(country = "GER", gender = "M", season_end_year = "2025", tier = "1")
```

Goalkeeper Ratings

```
##-----Goalkeeper-----
#>Have to divide this into Conventional Keeper Rating (CKR) and Sweeper Keeper Rating (SKR)
#>Take the keepers away from the playing times dataframe

gk_playing_time <- big5_player_playing_time %>% filter(Pos == "GK") %>% filter(MP_Playing.Time != 0)

df_list <- list(gk_playing_time, big5_player_keepers, big5_player_keepers_adv)
common_columns <- Reduce(intersect, lapply(df_list, colnames))

merged_df <- inner_join(big5_player_keepers, big5_player_keepers_adv, gk_playing_time, by = c("Season_E"))

gk_above_900 <- merged_df %>% filter(Min_Playing >= 900)

keepers_numeric <- gk_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
keepers_numeric <- keepers_numeric[, -1]
keepers_numeric <- na.omit(keepers_numeric)

# Scale the data (important for PCA since units differ)
keepers_scaled <- scale(keepers_numeric)
keepers_scaled <- keepers_scaled[, -1]
# Run PCA
keepers_pca <- PCA(keepers_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
keepers_pca$eig
```

```
##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  1.687747e+01          3.750548e+01          37.50548
## comp 2  6.462455e+00          1.436101e+01          51.86650
## comp 3  3.714958e+00          8.255461e+00          60.12196
## comp 4  2.702702e+00          6.006005e+00          66.12796
## comp 5  2.282551e+00          5.072336e+00          71.20030
## comp 6  1.987306e+00          4.416235e+00          75.61653
## comp 7  1.700050e+00          3.777888e+00          79.39442
## comp 8  1.351462e+00          3.003248e+00          82.39767
## comp 9  1.130311e+00          2.511803e+00          84.90947
## comp 10 9.948040e-01          2.210676e+00          87.12015
## comp 11 9.784206e-01          2.174268e+00          89.29441
## comp 12 9.014594e-01          2.003243e+00          91.29766
## comp 13 7.709435e-01          1.713208e+00          93.01087
## comp 14 5.702535e-01          1.267230e+00          94.27810
## comp 15 4.499212e-01          9.998249e-01          95.27792
## comp 16 4.009647e-01          8.910328e-01          96.16895
## comp 17 2.960463e-01          6.578807e-01          96.82683
## comp 18 2.817146e-01          6.260324e-01          97.45287
## comp 19 2.281545e-01          5.070100e-01          97.95988
## comp 20 1.675604e-01          3.723565e-01          98.33223
## comp 21 1.634280e-01          3.631734e-01          98.69541
## comp 22 1.086349e-01          2.414108e-01          98.93682
```

```
## comp 23 9.761297e-02      2.169177e-01      99.15373
## comp 24 8.283055e-02      1.840679e-01      99.33780
## comp 25 7.502015e-02      1.667115e-01      99.50451
## comp 26 6.162670e-02      1.369482e-01      99.64146
## comp 27 3.798040e-02      8.440088e-02      99.72586
## comp 28 3.271373e-02      7.269718e-02      99.79856
## comp 29 2.960309e-02      6.578464e-02      99.86435
## comp 30 1.518210e-02      3.373801e-02      99.89808
## comp 31 1.320659e-02      2.934799e-02      99.92743
## comp 32 1.200321e-02      2.667379e-02      99.95410
## comp 33 7.471943e-03      1.660432e-02      99.97071
## comp 34 6.194990e-03      1.376664e-02      99.98448
## comp 35 4.866255e-03      1.081390e-02      99.99529
## comp 36 1.307675e-03      2.905945e-03      99.99820
## comp 37 4.725365e-04      1.050081e-03      99.99925
## comp 38 2.741019e-04      6.091153e-04      99.99985
## comp 39 6.210513e-05      1.380114e-04      99.99999
## comp 40 2.495209e-06      5.544909e-06      100.00000
## comp 41 6.898508e-07      1.533002e-06      100.00000
## comp 42 1.425345e-31      3.167434e-31      100.00000
## comp 43 1.425345e-31      3.167434e-31      100.00000
## comp 44 1.425345e-31      3.167434e-31      100.00000
## comp 45 1.425345e-31      3.167434e-31      100.00000
```

```
# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- keepers_pca$eig[1, "percentage of variance"]
dim2_variance <- keepers_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

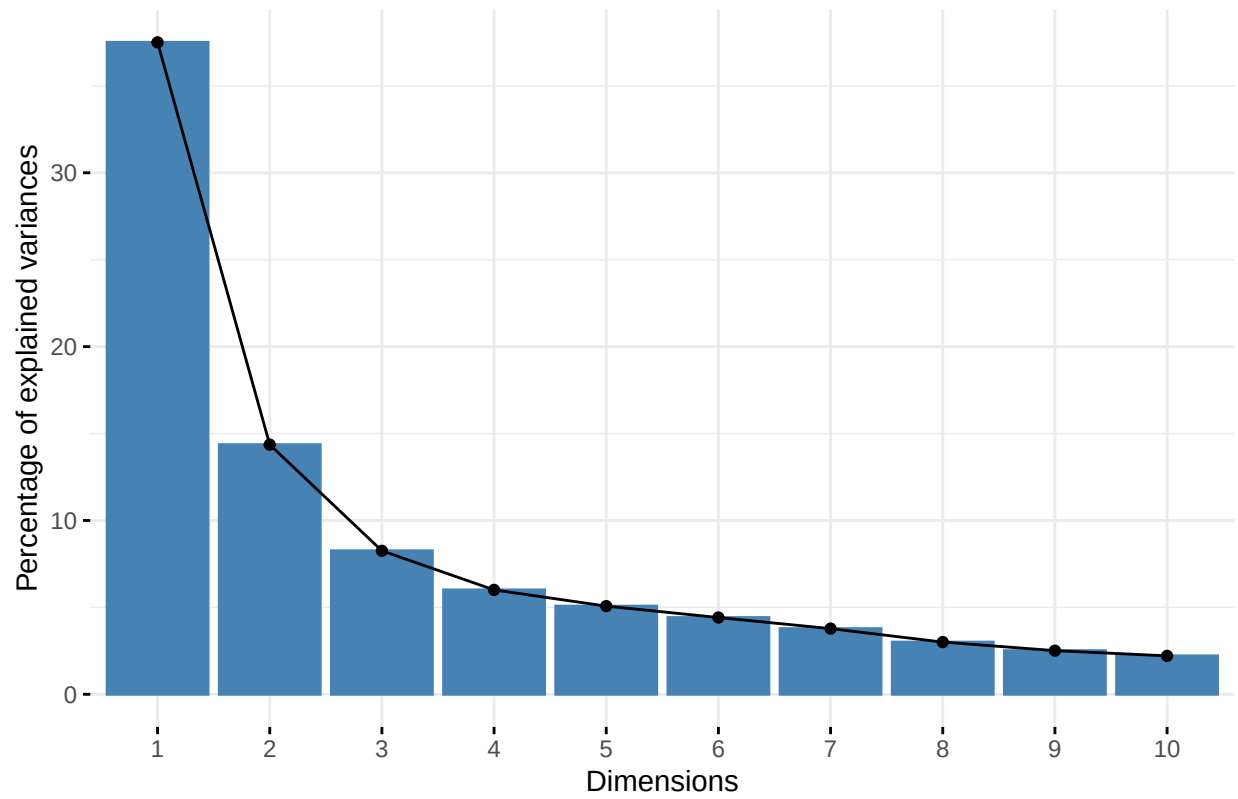
```
## [1] "Dim.1 weight: 0.723"
```

```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.277"
```

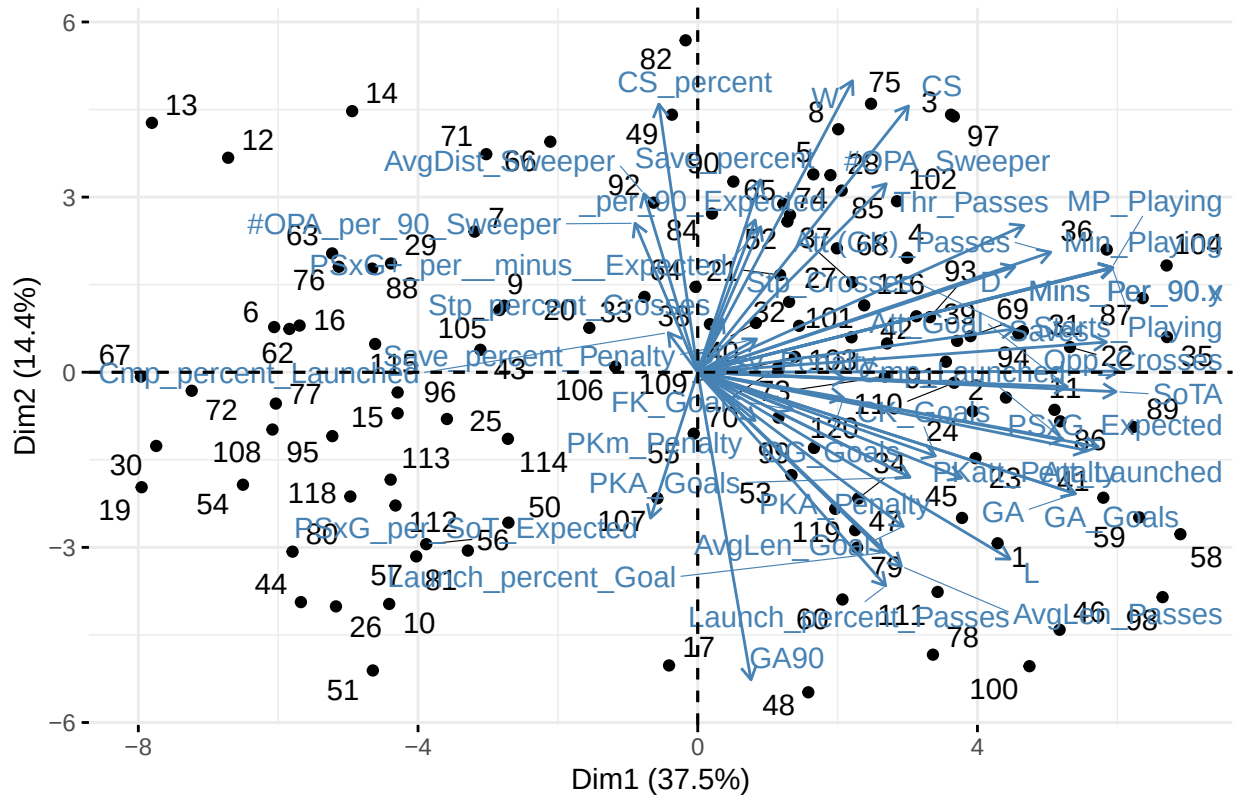
```
# Plot the variance explained
fviz_eig(keepers_pca)
```

Scree plot



```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(keepers_pca, repel = TRUE)
```

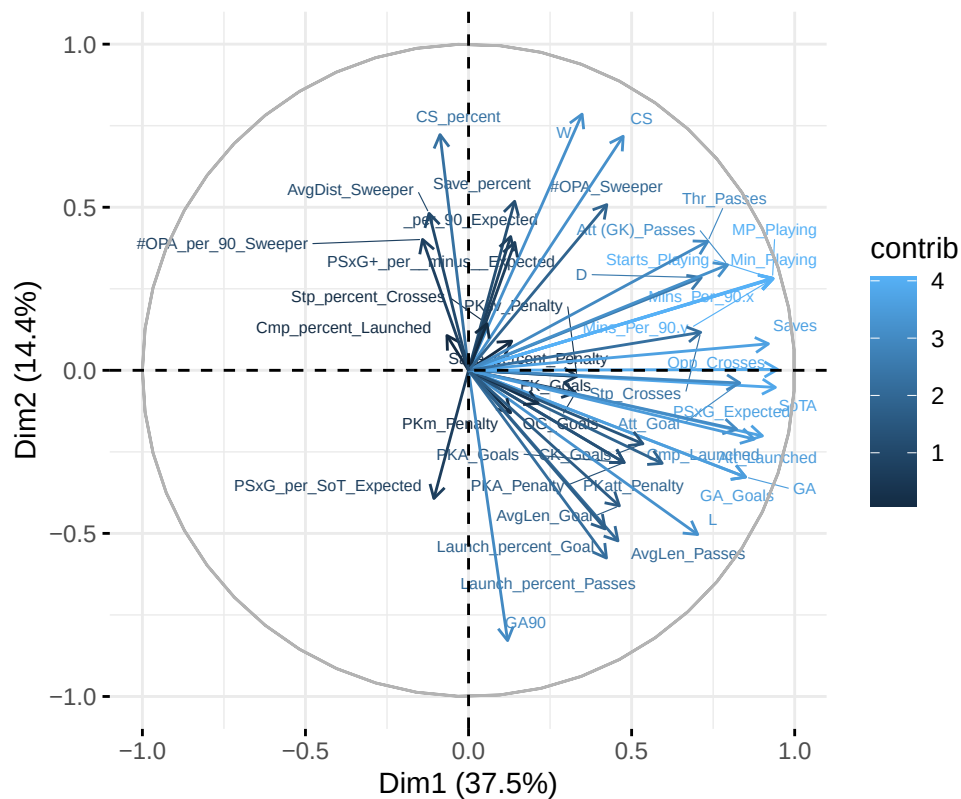
PCA – Biplot



Contributions of variables to PC1 and PC2

```
fviz_pca_var(keepers_pca,
             col.var = "contrib",
             labelsiz = 2, repel = TRUE)
```

Variables – PCA



```
variable_contrib <- data.frame(keepers_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

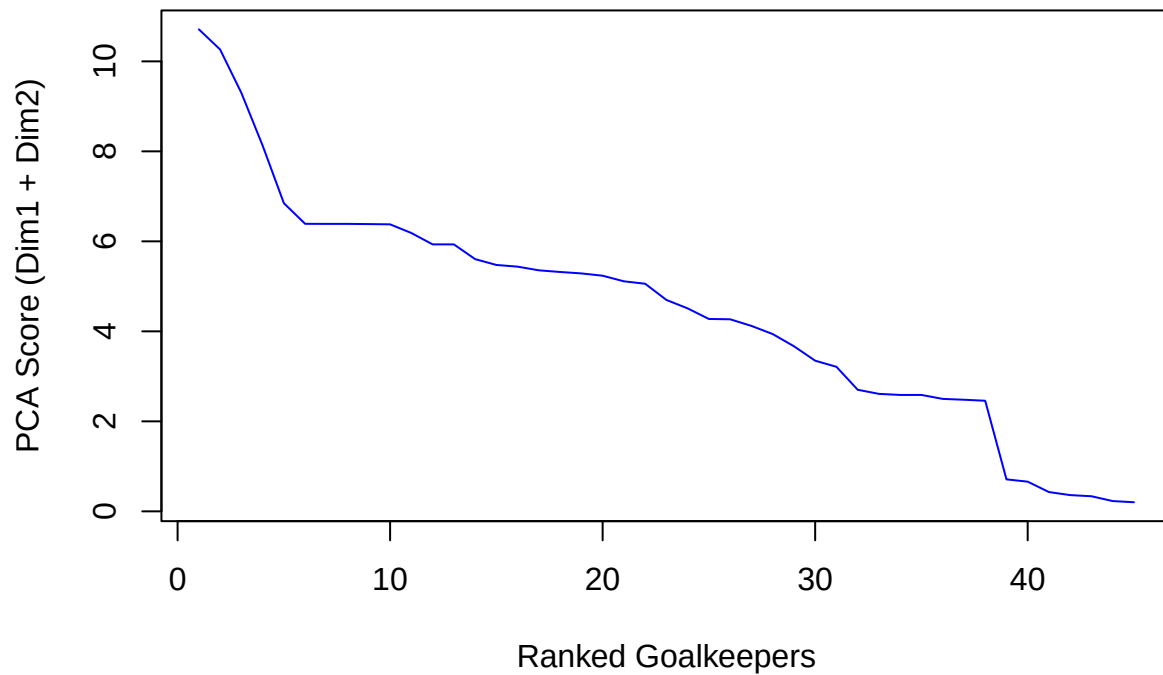
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 5.932545
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

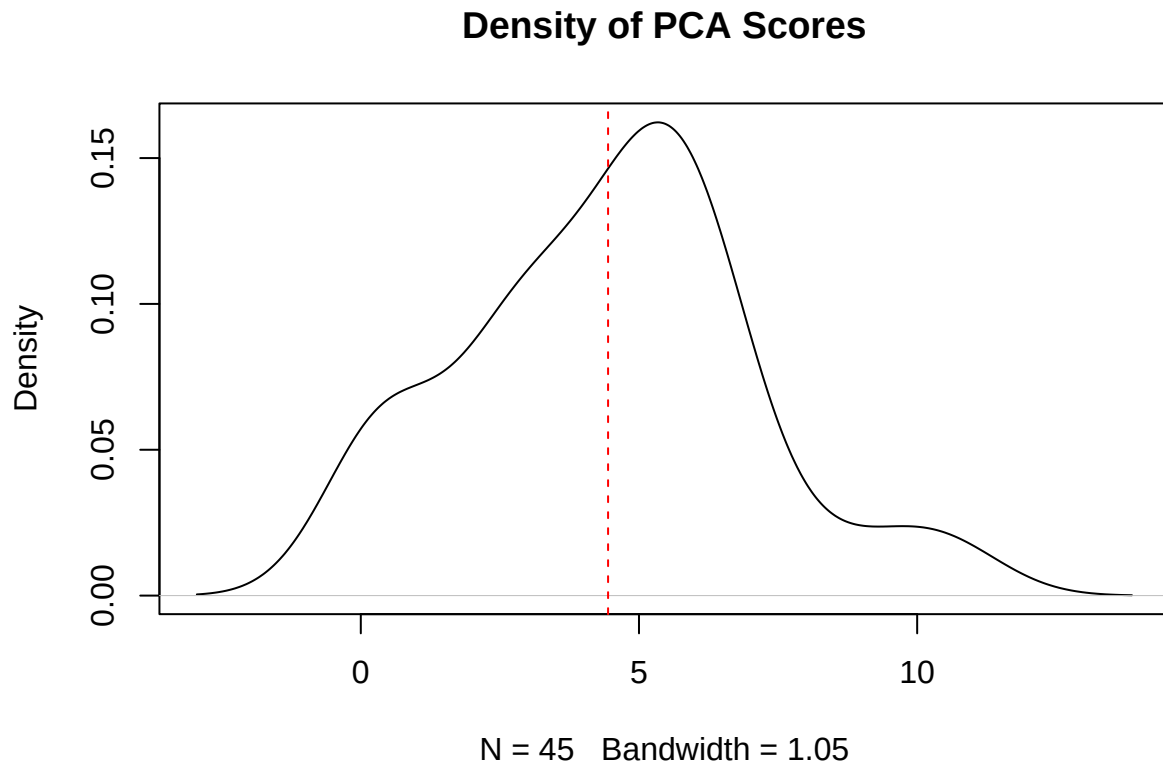
```
##      j1 j2 chi
## ESE  6 38  22
## EDE  6 38  22
```

```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 7.020209
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```

The PCA variable contribution plot shows a clear differentiation between shot stopping and sweeper keeper metrics

1. OPA_per_90_Sweeper and AvgDist_Sweeper are almost overlapping and long - These two are very closely related and both contribute heavily to PC1. TL;DR: your sweeper keeper stats move together and drive a big chunk of the differences between keepers.
2. PSxG+_per__minus__Expected (the expected goals prevented stat) points in a different direction and is quite long too - Strongly influential but not correlated with the sweeper actions. It's carrying weight independently, probably defining a classic shot-stopper profile.
3. Cmp_percent_Launched and Stp_percent_Crosses have shorter vectors, contributing less to the variance on these first two components. Could mean they're either:
 - Less variable between your keepers
 - Or important on other PCs (PC3, PC4...)

```
CKR <- with(gk_above_900,
  (Min_Playing *
    Saves * variable_contrib["Saves", "Weight"] *
    PSxG_Expected * variable_contrib["PSxG_Expected", "Weight"]) /
  (GA * variable_contrib["GA", "Weight"] * GA * SoTA * variable_contrib["SoTA", "Weight"])
)

SKR <- with(gk_above_900,
```

```
(
  (Cmp_Launched/Att_Launched) * variable_contrib["Cmp_Launched", "Weight"] +
  (Stp_Crosses/Opp_Crosses) * variable_contrib["Stp_Crosses", "Weight"] +
  (AvgLen_Passes/~ Att (GK)_Passes`) * variable_contrib["AvgLen_Passes", "Weight"] +
  (AvgDist_Sweeper/~ #OPA_Sweeper`) * variable_contrib["AvgDist_Sweeper", "Weight"] +
  (AvgLen_Goal/Att_Goal) * variable_contrib["AvgLen_Goal", "Weight"]
) / Min_Playing
)

avg_CKR <- mean(CKR, na.rm = TRUE)
avg_SKR <- mean(SKR, na.rm = TRUE)

CKR_Avg <- CKR / avg_CKR
SKR_Avg <- SKR / avg_SKR

Total_GK_Rating <- (CKR_Avg * dim1_weight) + (SKR_Avg * dim2_weight)

gk_ratings_final <- gk_above_900 %>%
  mutate(
    CKR_Avg = CKR_Avg,
    SKR_Avg = SKR_Avg,
    Total_GK_Rating = Total_GK_Rating
  ) %>%
  arrange(desc(Total_GK_Rating))

gk_ratings_24starts <- gk_ratings_final %>% filter(Starts_Playing >=25) %>% select(Squad, Comp, Player,
print(head(gk_ratings_24starts))
```

```
##           Squad           Comp           Player  CKR_Avg  SKR_Avg
## 1           Roma           Serie A           Mile Svilar 1.939451 0.3573098
## 2           Napoli           Serie A           Alex Meret 1.802390 0.4190483
## 3           Inter           Serie A           Yann Sommer 1.686317 0.6603427
## 4 Manchester City Premier League           Ederson 1.475825 0.6777597
## 5           Lille           Ligue 1 Lucas Chevalier 1.518066 0.5338584
## 6           Arsenal Premier League           David Raya 1.569942 0.3685976
##   Total_GK_Rating
## 1           1.501382
## 2           1.419365
## 3           1.402241
## 4           1.254853
## 5           1.245555
## 6           1.237309
```

```
#Correlation work between GA and GKR
```

```
cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$GA, use = "complete.obs", method = "spearman")
```

```
## [1] -0.8340638
```

```
cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$`PSxG+_per__minus__Expected`, use = "complete.obs")
```

```
## [1] 0.478998
```

```
cor(gk_ratings_final$Total_GK_Rating, (gk_ratings_final$PSxG_Expected - gk_ratings_final$GA) / (gk_ratings_final$PSxG_Expected + gk_ratings_final$GA))
```

```
## [1] 0.5381741
```

```
cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$Save_percent, use = "complete.obs", method = "spearmanr")
```

```
## [1] 0.6137991
```

```
gk_ratings_league_avg <- gk_ratings_final %>% group_by(Comp) %>% summarise(Avg_GKR = mean(Total_GK_Rating))
print(gk_ratings_league_avg)
```

```
## # A tibble: 5 x 2
##   Comp      Avg_GKR
##   <chr>      <dbl>
## 1 Bundesliga 0.885
## 2 La Liga    1.03
## 3 Ligue 1    0.985
## 4 Premier League 0.998
## 5 Serie A    1.08
```

```
gk_psxg_league_avg <- gk_ratings_final %>% group_by(Comp) %>% summarise(Avg_PsxG = mean(PSxG_Expected))
print(gk_psxg_league_avg)
```

```
## # A tibble: 5 x 2
##   Comp      Avg_PsxG
##   <chr>      <dbl>
## 1 Bundesliga 38.8
## 2 La Liga    33.9
## 3 Ligue 1    37.9
## 4 Premier League 37.3
## 5 Serie A    32.5
```

```
cor(gk_ratings_league_avg$Avg_GKR, gk_psxg_league_avg$Avg_PsxG, method = "spearmanr")
```

```
## [1] -1
```

```
#Split them into Each League
```

```
pl_gk_ratings <- gk_ratings_final %>% filter(Comp == "Premier League")
bundesliga_gk_ratings <- gk_ratings_final %>% filter(Comp == "Bundesliga")
seriea_gk_ratings <- gk_ratings_final %>% filter(Comp == "Serie A")
ligue1_gk_ratings <- gk_ratings_final %>% filter(Comp == "Ligue 1")
laliga_gk_ratings <- gk_ratings_final %>% filter(Comp == "La Liga")
```

```
#Awards for each league
```

```
pl_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Premier League")
bundesliga_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Bundesliga")
seriea_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Serie A")
ligue1_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Ligue 1")
laliga_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "La Liga")
```

Interpretation

These correlations confirm that the model is strongly aligned with key outcome metrics while maintaining balance:

High correlation with Goals Against indicates outcome relevance.

Moderate correlations with PSxG-GA per 90 and Save Percentage confirm the model's shot-stopping validity without overfitting to a single dimension.

The moderate strength of these relationships reflects the rating's inclusion of modern, non-shot-stopping actions, enhancing its relevance to contemporary goalkeeper profiles.

Conclusion

This multidimensional rating offers a more comprehensive view of goalkeeper performance than conventional single-metric models. It captures the hybrid nature of the modern goalkeeper — one expected to contribute both in traditional shot-stopping and proactive defensive play — and stands up well to statistical validation against key performance outcomes.

Outfield Player Ratings

Defensive Player Ratings

```
#>DEFENSIVE PLAYER RATING
```

```
outfield_playing_time <- big5_player_playing_time %>% filter(Pos != "GK") %>% filter(Min_Playing.Time > 900)
outfield_df_list <- list(outfield_playing_time, big5_player_defense, big5_player_gca, big5_player_misc,
                        big5_player_passing, big5_player_passing_types, big5_player_possession,
                        big5_player_shooting, big5_player_standard)
common_columns_outfield <- Reduce(intersect, lapply(outfield_df_list, colnames))
print(common_columns_outfield)
```

```
## [1] "Season_End_Year" "Squad"          "Comp"          "Player"
## [5] "Nation"          "Pos"           "Age"           "Born"
## [9] "Url"
```

```
outfield_data <- outfield_df_list %>%
  reduce(left_join, by = c("Season_End_Year", "Squad", "Comp", "Player", "Nation", "Pos", "Age", "Born"))
outfield_data <- outfield_data %>%
  select(-Url)
```

```
outfield_defensive_data <- outfield_data %>% select(Season_End_Year, Squad, Comp, Player, Nation, Pos, Age, Born)
```

```
outfield_above_900 <- outfield_defensive_data %>% filter(Min_Playing.Time >= 900)
```

```
outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))
```

```
# Optional: remove NAs
```

```
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)
```

```

# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[, -1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig

```

```

##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  6.64820848          44.3213899          44.32139
## comp 2  2.13797032          14.2531355          58.57453
## comp 3  1.65014735          11.0009824          69.57551
## comp 4  1.02442469           6.8294979          76.40501
## comp 5  0.83344331           5.5562887          81.96129
## comp 6  0.67132965           4.4755310          86.43683
## comp 7  0.52666543           3.5111029          89.94793
## comp 8  0.33063742           2.2042495          92.15218
## comp 9  0.32238425           2.1492283          94.30141
## comp 10 0.28455172           1.8970114          96.19842
## comp 11 0.20232377           1.3488251          97.54724
## comp 12 0.14588291           0.9725527          98.51980
## comp 13 0.14318506           0.9545670          99.47436
## comp 14 0.05038787           0.3359191          99.81028
## comp 15 0.02845778           0.1897185         100.00000

```

```

# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))

```

```
## [1] "Dim.1 weight: 0.757"
```

```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

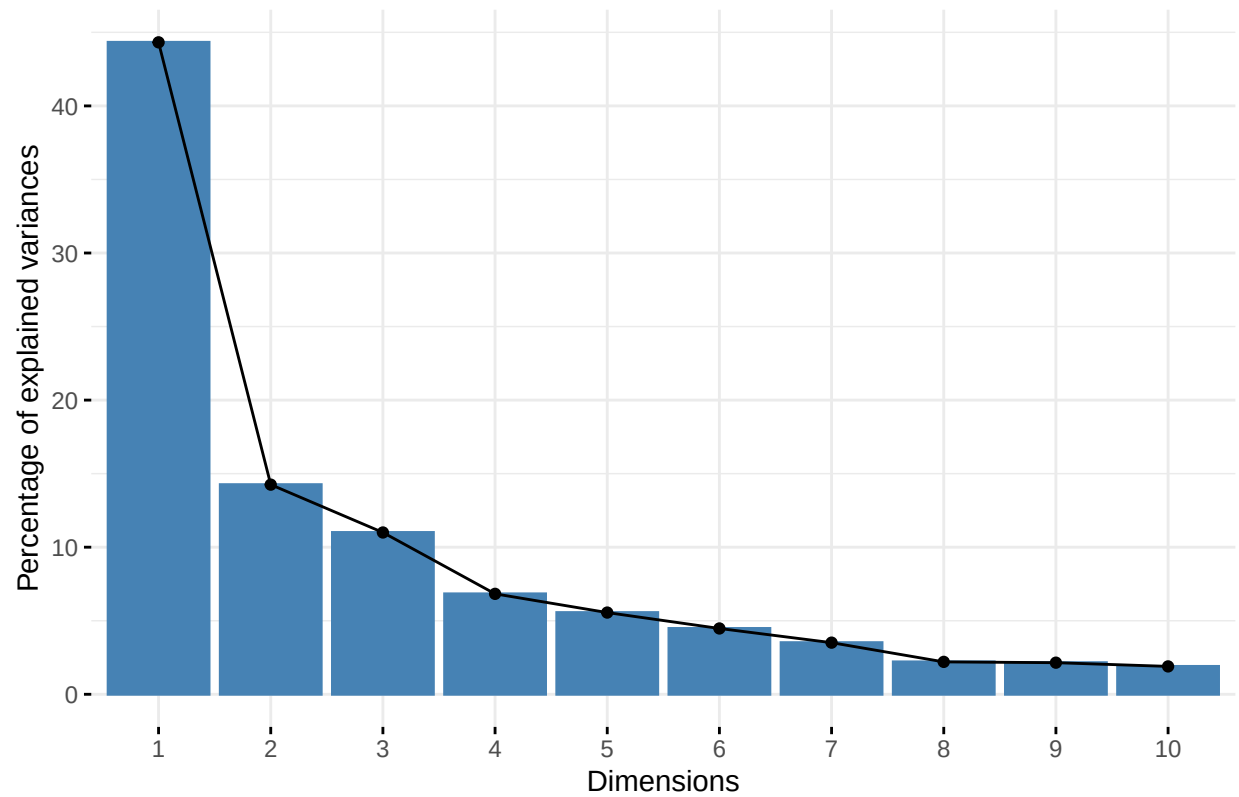
```
## [1] "Dim.2 weight: 0.243"
```

```

# Plot the variance explained
fviz_eig(outfield_pca)

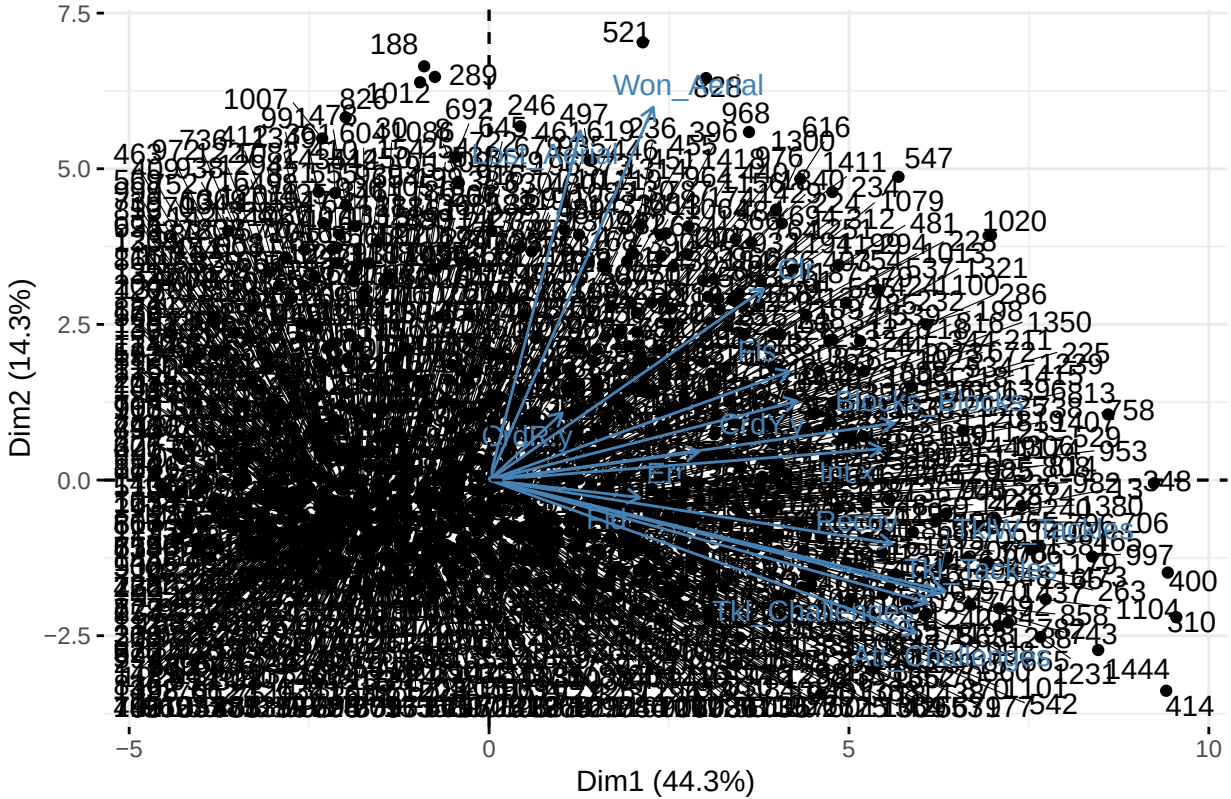
```

Scree plot

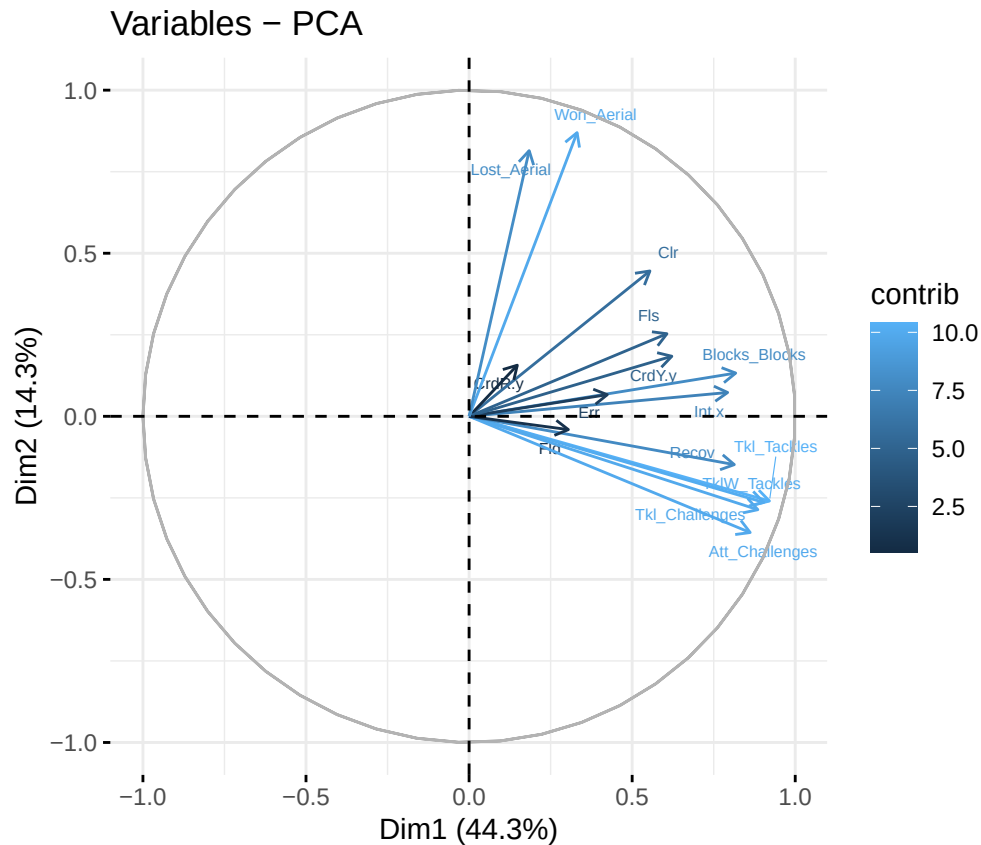


```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```

PCA – Biplot



```
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
  col.var = "contrib",
  labelsize = 2, repel = TRUE)
```



```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

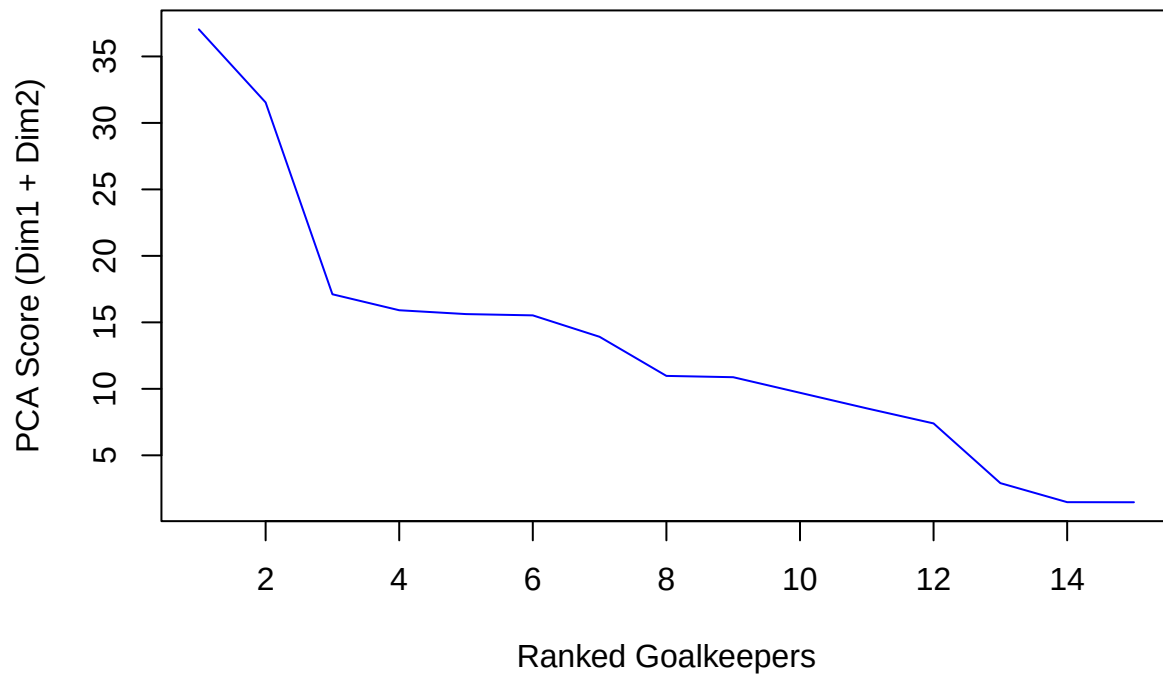
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 15.76682
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
      ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```


PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##      j1 j2 chi
## ESE  3 12 7.5
## EDE  3  1 NaN
```

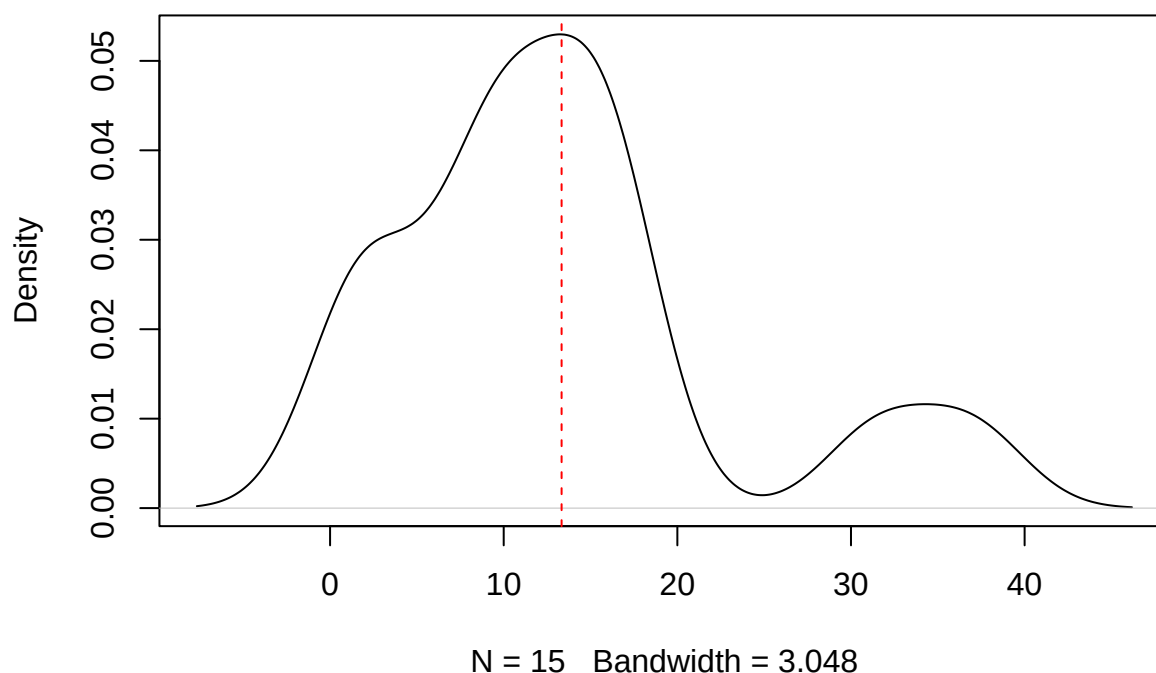
```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 23.32487
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```

Density of PCA Scores



#####

```
outfield_dpr <- outfield_data %>% mutate(Defensive_Player_Rtg = ((Tk1_Tackles* variable_contrib["Tk1_Tackles"])/
  variable_contrib["Tk1_Tackles"]))

#> Couple of values have -Inf, NA or NaN for DPR. Have to take them out - Row 555, 833, 728, 1408
#> Have to change this to a code so it does it automatically
outfield_dpr_filtered <- outfield_dpr %>%
  filter(!row_number() %in% c(417, 547, 1112, 1386))

#> Get the Average possession for every team in the top 5 leagues (will be useful for other metrics too)
#> For defenders, divide DPR by (100 - Avg. Poss.)
#> First things first, let's extract the possession data at team level

big5_team_possession <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "possession", t
dplyr::glimpse(big5_team_possession)
```

```
## Rows: 192
## Columns: 30
## $ Season_End_Year      <int> 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2025, 20~
## $ Squad                <chr> "Alavés", "Alavés", "Angers", "Angers", "Arsenal",~
## $ Comp                 <chr> "La Liga", "La Liga", "Ligue 1", "Ligue 1", "Premi~
## $ Team_or_Opponent     <chr> "team", "opponent", "team", "opponent", "team", "o~
## $ Num_Players          <dbl> 29, 29, 30, 30, 25, 25, 28, 28, 32, 32, 31, 31, 24~
```

```
## $ Poss <dbl> 45.3, 54.7, 41.2, 58.8, 56.9, 43.1, 50.5, 49.5, 55~
## $ Mins_Per_90 <dbl> 38, 38, 34, 34, 38, 38, 38, 38, 38, 38, 38, 38~
## $ Touches_Touches <dbl> 19070, 22689, 17919, 24043, 24056, 19306, 21699, 2~
## $ 'Def Pen_Touches' <dbl> 1876, 2367, 2146, 1747, 1844, 2948, 2646, 2408, 20~
## $ 'Def 3rd_Touches' <dbl> 5802, 7584, 6594, 5934, 6064, 7642, 7301, 6804, 69~
## $ 'Mid 3rd_Touches' <dbl> 8634, 10446, 7732, 11764, 10362, 7501, 9257, 9497, ~
## $ 'Att 3rd_Touches' <dbl> 4809, 4876, 3813, 6568, 7808, 4397, 5362, 5515, 70~
## $ 'Att Pen_Touches' <dbl> 691, 659, 512, 921, 1282, 632, 951, 886, 1076, 555~
## $ Live_Touches <dbl> 19059, 22678, 17913, 24038, 24054, 19303, 21693, 2~
## $ Att_Take <dbl> 478, 810, 613, 649, 726, 595, 726, 617, 645, 573, ~
## $ Succ_Take <dbl> 185, 378, 249, 313, 320, 255, 327, 236, 257, 234, ~
## $ Succ_percent_Take <dbl> 38.7, 46.7, 40.6, 48.2, 44.1, 42.9, 45.0, 38.2, 39~
## $ Tkld_Take <dbl> 245, 348, 331, 294, 316, 260, 299, 310, 330, 272, ~
## $ Tkld_percent_Take <dbl> 51.3, 43.0, 54.0, 45.3, 43.5, 43.7, 41.2, 50.2, 51~
## $ Carries_Carries <dbl> 9128, 11958, 9398, 14633, 14234, 10350, 12334, 120~
## $ TotDist_Carries <dbl> 45578, 63721, 51759, 79121, 75544, 57925, 68906, 6~
## $ PrgDist_Carries <dbl> 22946, 33531, 26951, 42638, 40246, 30357, 36242, 3~
## $ PrgC_Carries <dbl> 449, 573, 502, 784, 852, 618, 720, 688, 897, 510, ~
## $ Final_Third_Carries <dbl> 346, 440, 365, 623, 540, 413, 448, 462, 696, 355, ~
## $ CPA_Carries <dbl> 90, 137, 98, 185, 286, 162, 212, 205, 254, 115, 18~
## $ Mis_Carries <dbl> 598, 645, 493, 469, 483, 451, 508, 491, 574, 603, ~
## $ Dis_Carries <dbl> 307, 389, 297, 330, 382, 338, 386, 327, 362, 299, ~
## $ Rec_Receiving <dbl> 10419, 14127, 10991, 17198, 16894, 12077, 14423, 1~
## $ PrgR_Receiving <dbl> 1148, 1309, 943, 1756, 1801, 927, 1331, 1292, 1808~
## $ Url <chr> "https://fbref.com/en/squads/8d6fd021/Alaves-Stats~
```

```
big5_team_possession_opponent <- big5_team_possession %>% filter(Team_or_Opponent == "opponent") %>% se
big5_team_possession_team <- big5_team_possession %>% filter(Team_or_Opponent == "team") %>% select(Squa

outfield_dpr_filtered_possopp <- outfield_dpr_filtered %>% left_join(big5_team_possession_opponent, by =
outfield_dpr_filtered_possopp_possteam <- outfield_dpr_filtered_possopp %>% left_join(big5_team_possess

outfield_DPR <- outfield_dpr_filtered_possopp_possteam %>% mutate(Poss.Adj.DPR = Defensive_Player_Rtg*1
avg_dpr <- mean(outfield_DPR$Poss.Adj.DPR)
outfield_DPR <- outfield_DPR %>% mutate(Poss.Adj.DPR = (Poss.Adj.DPR/avg_dpr)) %>% arrange(desc(Poss.Adj

pl_dpr <- outfield_DPR %>% filter(Comp == "Premier League")
ligue1_dpr <- outfield_DPR %>% filter(Comp == "Ligue 1")
seriea_dpr <- outfield_DPR %>% filter(Comp == "Serie A")
bundesliga_dpr <- outfield_DPR %>% filter(Comp == "Bundesliga")
laliga_dpr <- outfield_DPR %>% filter(Comp == "La Liga")
```

#> Outfield_DPR is what you have to carry forward because you cannot have players ineligible for any ca

Offensive Creation Rating

#>OFFENSIVE CREATION RATING

```
outfield_ocr_data <- outfield_data %>% select(Season_End_Year, Squad, Comp, Player, Nation, Pos, Age, M
colnames(outfield_ocr_data)[21] <- "Att_3rd_Touches"
```

```

colnames(outfield_ocr_data)[22] <- "Att_Pen_Touches"

outfield_above_900 <- outfield_ocr_data %>% filter(Min_Playing.Time >= 900)
outfield_above_900_complete <- na.omit(outfield_above_900)

outfield_numeric <- outfield_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)

# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[, -1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig

```

```

##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  13.340153694          47.643406051          47.64341
## comp 2   7.837584325          27.991372588          75.63478
## comp 3   1.853327019           6.619025069          82.25380
## comp 4   1.148772616           4.102759341          86.35656
## comp 5   0.803868532           2.870959044          89.22752
## comp 6   0.604274419           2.158122924          91.38565
## comp 7   0.487217597           1.740062845          93.12571
## comp 8   0.371731659           1.327613068          94.45332
## comp 9   0.266017851           0.950063755          95.40338
## comp 10  0.227426354           0.812236980          96.21562
## comp 11  0.166037668           0.592991673          96.80861
## comp 12  0.138585750           0.494949108          97.30356
## comp 13  0.136748960           0.488389144          97.79195
## comp 14  0.109344401           0.390515719          98.18247
## comp 15  0.094169251           0.336318753          98.51879
## comp 16  0.084751537           0.302684062          98.82147
## comp 17  0.069564207           0.248443595          99.06991
## comp 18  0.064228018           0.229385779          99.29930
## comp 19  0.047424535           0.169373340          99.46867
## comp 20  0.037410009           0.133607174          99.60228
## comp 21  0.025930678           0.092609564          99.69489
## comp 22  0.022798958           0.081424851          99.77631
## comp 23  0.021751072           0.077682401          99.85400
## comp 24  0.014023205           0.050082873          99.90408
## comp 25  0.011723856           0.041870913          99.94595
## comp 26  0.008381414           0.029933620          99.97588
## comp 27  0.005269369           0.018819174          99.99470
## comp 28  0.001483046           0.005296592         100.00000

```

```

# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

```

```
# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

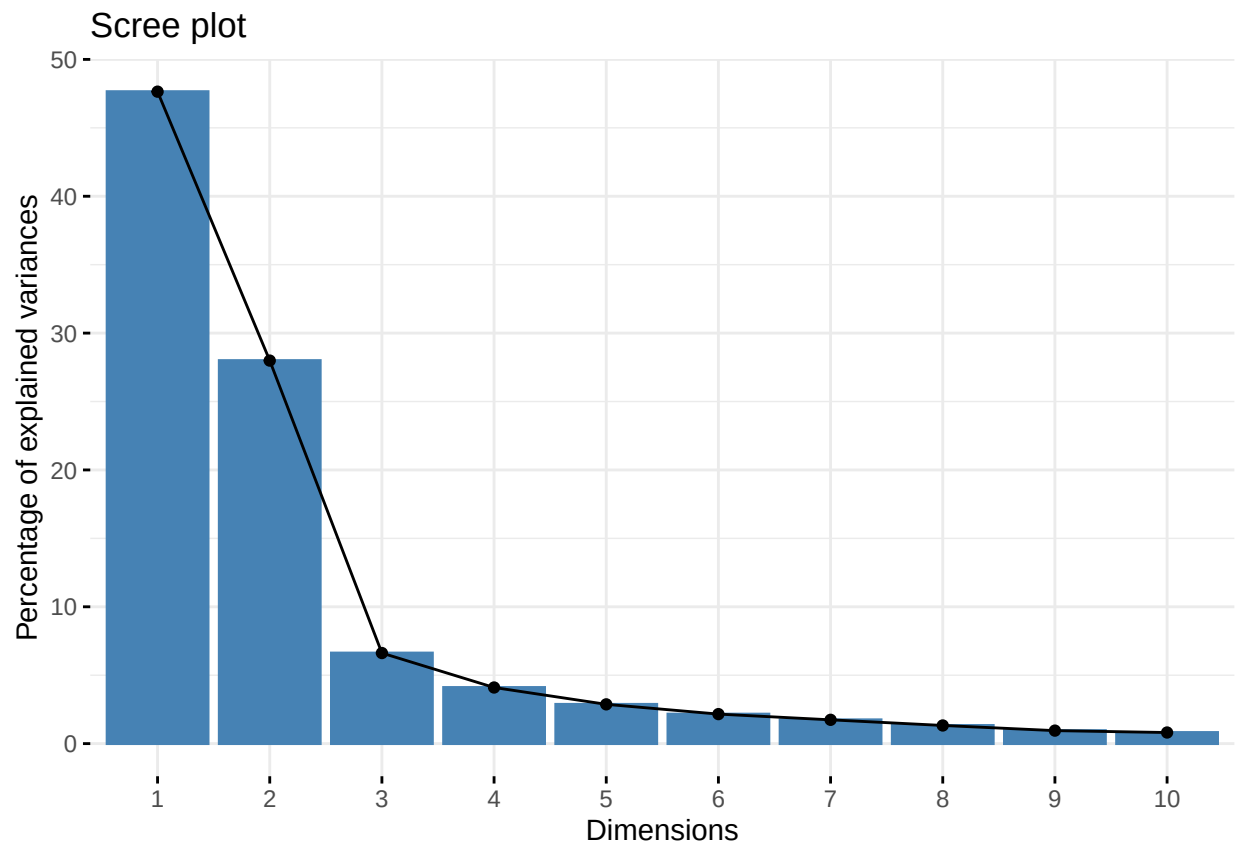
print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.63"
```

```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.37"
```

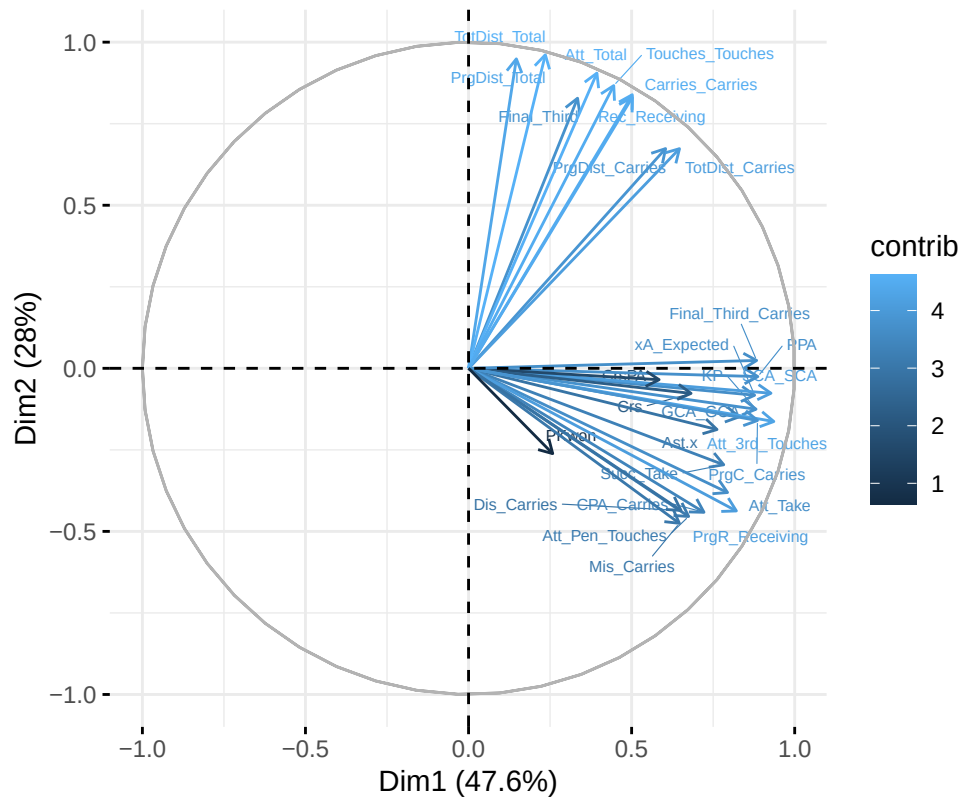
```
# Plot the variance explained
fviz_eig(outfield_pca)
```



```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```

```
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
             col.var = "contrib",
             labelsize = 2, repel = TRUE)
```

Variables – PCA



```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

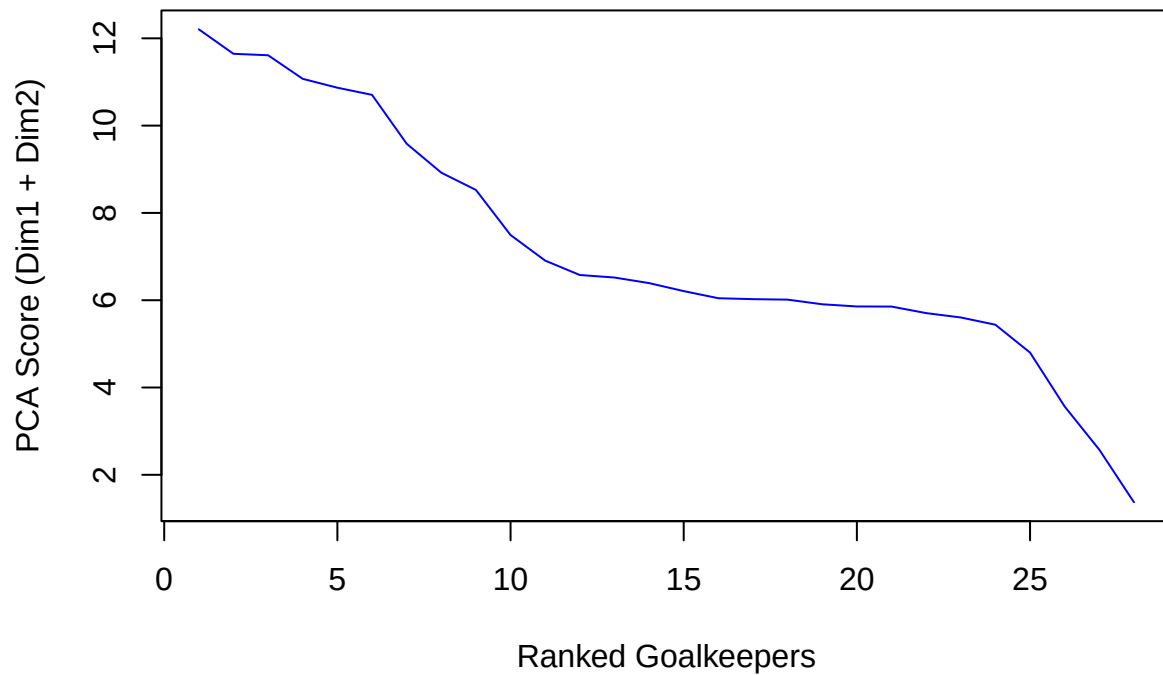
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 9.087791
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
      ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##      j1 j2  chi
## ESE 12 24 18.0
## EDE 11 24 17.5
```

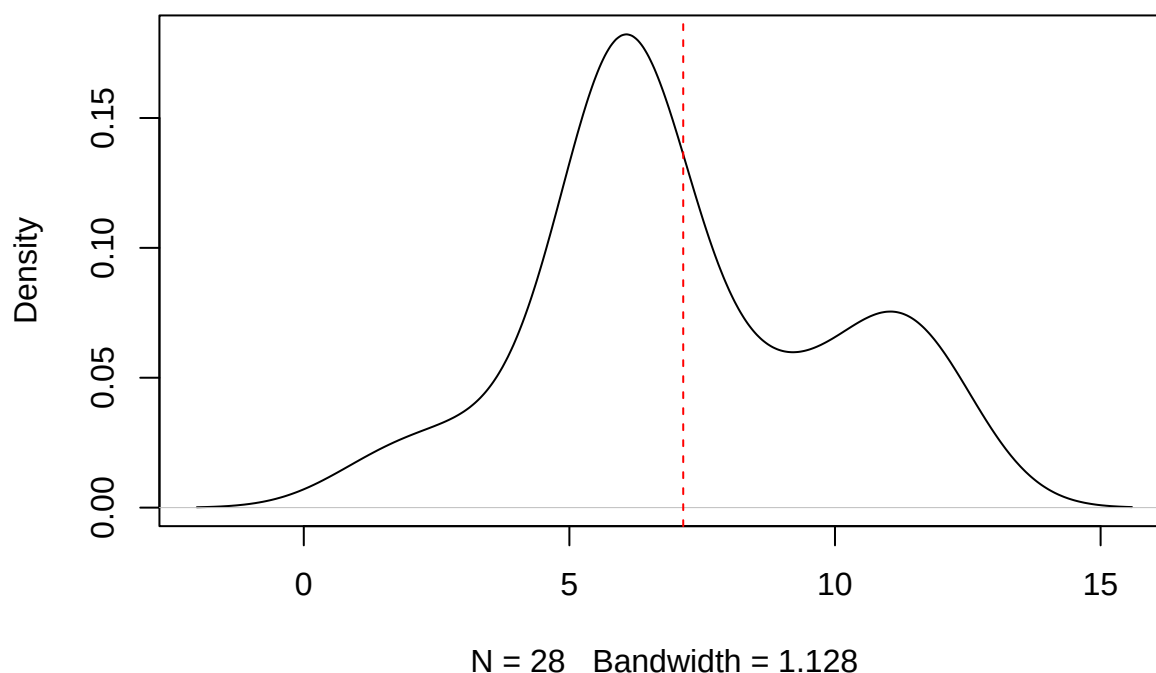
```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 9.929307
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```


Density of PCA Scores



#####

#> OFFENSIVE CREATION RATING

```
outfield_DPR_ocr <- outfield_above_900 %>% mutate(Off_Creation_Rtg = (((Ast.x *variable_contrib["Ast.x"]
setdiff(c("Ast.x", "xA_Expected", "SCA_SCA", "GCA_GCA", "KP", "Final_Third", "PPA", "Att_Total", "CrsPA
```

```
## character(0)
```

```
big5_team_possession <- fb_big5_advanced_season_stats(season_end_year= 2025, stat_type= "possession", t
dplyr::glimpse(big5_team_possession)
```

```
## Rows: 192
## Columns: 30
## $ Season_End_Year      <int> 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2025, 20~
## $ Squad                <chr> "Alavés", "Alavés", "Angers", "Angers", "Arsenal",~
## $ Comp                 <chr> "La Liga", "La Liga", "Ligue 1", "Ligue 1", "Premi~
## $ Team_or_Opponent     <chr> "team", "opponent", "team", "opponent", "team", "o~
## $ Num_Players          <dbl> 29, 29, 30, 30, 25, 25, 28, 28, 32, 32, 31, 31, 24~
## $ Poss                 <dbl> 45.3, 54.7, 41.2, 58.8, 56.9, 43.1, 50.5, 49.5, 55~
## $ Mins_Per_90          <dbl> 38, 38, 34, 34, 38, 38, 38, 38, 38, 38, 38, 38, 38~
## $ Touches_Touches      <dbl> 19070, 22689, 17919, 24043, 24056, 19306, 21699, 2~
## $ 'Def Pen_Touches'    <dbl> 1876, 2367, 2146, 1747, 1844, 2948, 2646, 2408, 20~
```

```
## $ 'Def 3rd_Touches' <dbl> 5802, 7584, 6594, 5934, 6064, 7642, 7301, 6804, 69~
## $ 'Mid 3rd_Touches' <dbl> 8634, 10446, 7732, 11764, 10362, 7501, 9257, 9497, ~
## $ 'Att 3rd_Touches' <dbl> 4809, 4876, 3813, 6568, 7808, 4397, 5362, 5515, 70~
## $ 'Att Pen_Touches' <dbl> 691, 659, 512, 921, 1282, 632, 951, 886, 1076, 555~
## $ Live_Touches <dbl> 19059, 22678, 17913, 24038, 24054, 19303, 21693, 2~
## $ Att_Take <dbl> 478, 810, 613, 649, 726, 595, 726, 617, 645, 573, ~
## $ Succ_Take <dbl> 185, 378, 249, 313, 320, 255, 327, 236, 257, 234, ~
## $ Succ_percent_Take <dbl> 38.7, 46.7, 40.6, 48.2, 44.1, 42.9, 45.0, 38.2, 39~
## $ Tkld_Take <dbl> 245, 348, 331, 294, 316, 260, 299, 310, 330, 272, ~
## $ Tkld_percent_Take <dbl> 51.3, 43.0, 54.0, 45.3, 43.5, 43.7, 41.2, 50.2, 51~
## $ Carries_Carries <dbl> 9128, 11958, 9398, 14633, 14234, 10350, 12334, 120~
## $ TotDist_Carries <dbl> 45578, 63721, 51759, 79121, 75544, 57925, 68906, 6~
## $ PrgDist_Carries <dbl> 22946, 33531, 26951, 42638, 40246, 30357, 36242, 3~
## $ PrgC_Carries <dbl> 449, 573, 502, 784, 852, 618, 720, 688, 897, 510, ~
## $ Final_Third_Carries <dbl> 346, 440, 365, 623, 540, 413, 448, 462, 696, 355, ~
## $ CPA_Carries <dbl> 90, 137, 98, 185, 286, 162, 212, 205, 254, 115, 18~
## $ Mis_Carries <dbl> 598, 645, 493, 469, 483, 451, 508, 491, 574, 603, ~
## $ Dis_Carries <dbl> 307, 389, 297, 330, 382, 338, 386, 327, 362, 299, ~
## $ Rec_Receiving <dbl> 10419, 14127, 10991, 17198, 16894, 12077, 14423, 1~
## $ PrgR_Receiving <dbl> 1148, 1309, 943, 1756, 1801, 927, 1331, 1292, 1808~
## $ Url <chr> "https://fbref.com/en/squads/8d6fd021/Alaves-Stats~
```

```
big5_team_possession_opponent <- big5_team_possession %>% filter(Team_or_Opponent == "opponent") %>% se
big5_team_possession_team <- big5_team_possession %>% filter(Team_or_Opponent == "team") %>% select(Squa

outfield_DPR_ocr <- outfield_DPR_ocr %>% left_join(big5_team_possession_opponent, by = "Squad", relation
outfield_DPR_ocr <- outfield_DPR_ocr %>% left_join(big5_team_possession_team, by = "Squad", relationship

outfield_DPR_ocr_possadj <- outfield_DPR_ocr %>% mutate(Poss.adj.OCR = (Off_Creation_Rtg/Poss_Team)*100
outfield_DPR_ocr_possadj <- outfield_DPR_ocr_possadj %>% filter(Poss.adj.OCR != "NaN")
avg_ocr <- mean(outfield_DPR_ocr_possadj$Poss.adj.OCR)
outfield_DPR_OCR <- outfield_DPR_ocr_possadj %>% mutate(Poss.adj.OCR = Poss.adj.OCR/avg_ocr) %>% arrang

outfield_DPR_OCR_5ast <- outfield_DPR_OCR %>% filter(Ast.x >= 5)
```

Goalscorer Rating

#>GOALSCORER RATING

```
outfield_gsr_data <- outfield_data %>%
  mutate(
    Goals_per_xG = ifelse(xG_Expected.x > 0, Gls_Standard / xG_Expected.x, 0),
    G_per_Shot = ifelse(Sh_Standard > 0, Gls_Standard / Sh_Standard, 0),
    xG_per_Shot = ifelse(Sh_Standard > 0, xG_Expected.x / Sh_Standard, 0))

outfield_gsr_data <- outfield_gsr_data %>% mutate(Gls_per_xG = Gls/xG_Expected.x) %>% select(Season_End,

colnames(outfield_gsr_data)[10] <- "xG_Expected"

outfield_above_900 <- outfield_gsr_data %>% filter(Min_Playing.Time >= 900)
outfield_above_900_complete <- na.omit(outfield_above_900)
```

```
outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))
```

```
# Optional: remove NAs
```

```
outfield_numeric <- outfield_numeric[, -1]
```

```
outfield_numeric <- na.omit(outfield_numeric)
```

```
# Scale the data (important for PCA since units differ)
```

```
outfield_scaled <- scale(outfield_numeric)
```

```
outfield_scaled <- outfield_scaled[, -1]
```

```
# Run PCA
```

```
outfield_pca <- PCA(outfield_scaled, graph = FALSE)
```

```
# View eigenvalues (variance explained by each component)
```

```
outfield_pca$eig
```

```
##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  5.942115e+00          5.401922e+01          54.01922
## comp 2  2.375944e+00          2.159949e+01          75.61871
## comp 3  1.076489e+00          9.786266e+00          85.40498
## comp 4  7.127309e-01          6.479372e+00          91.88435
## comp 5  4.326939e-01          3.933581e+00          95.81793
## comp 6  1.966079e-01          1.787345e+00          97.60528
## comp 7  1.437517e-01          1.306834e+00          98.91211
## comp 8  7.090490e-02          6.445900e-01          99.55670
## comp 9  4.396590e-02          3.996900e-01          99.95639
## comp 10 4.796876e-03          4.360796e-02          100.00000
## comp 11 1.344854e-31          1.222595e-30          100.00000
```

```
# Extract variance explained by Dim.1 and Dim.2
```

```
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
```

```
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]
```

```
# Normalize so they sum to 1 (as relative weights)
```

```
total_variance <- dim1_variance + dim2_variance
```

```
dim1_weight <- dim1_variance / total_variance
```

```
dim2_weight <- dim2_variance / total_variance
```

```
print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.714"
```

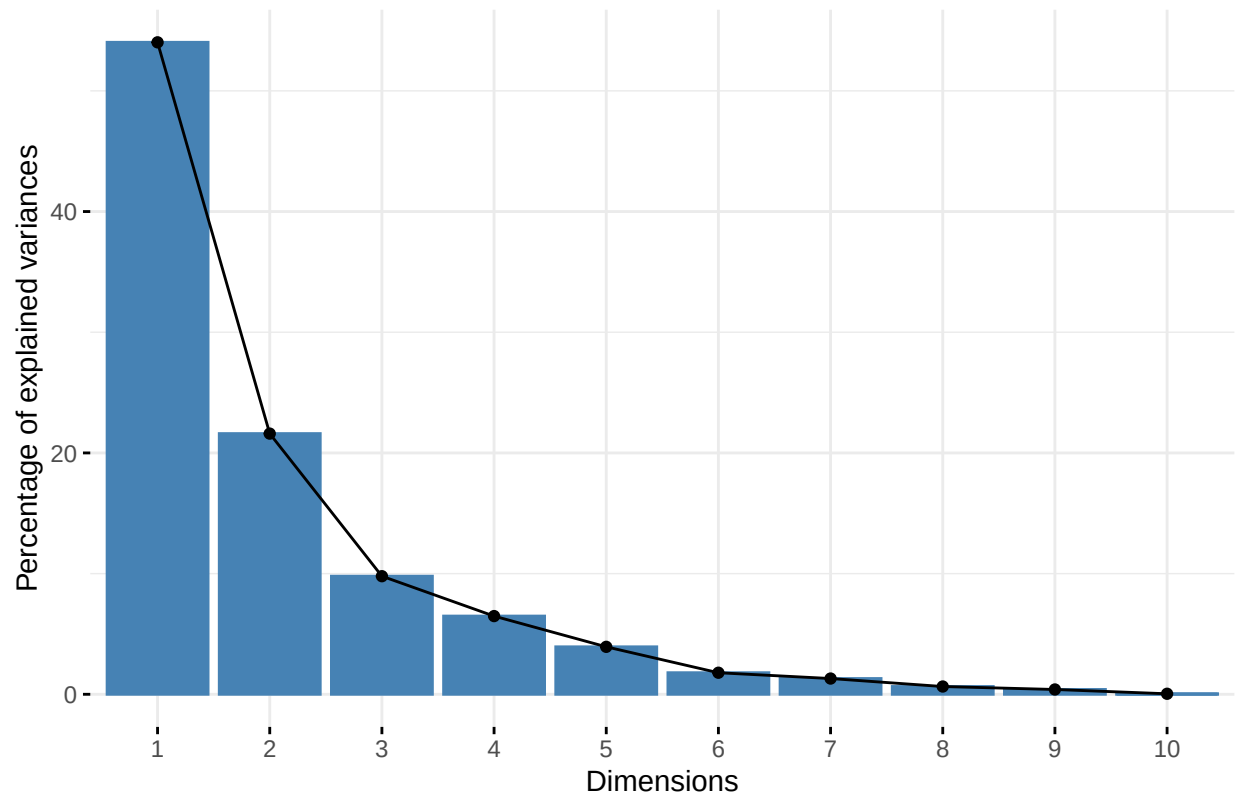
```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.286"
```

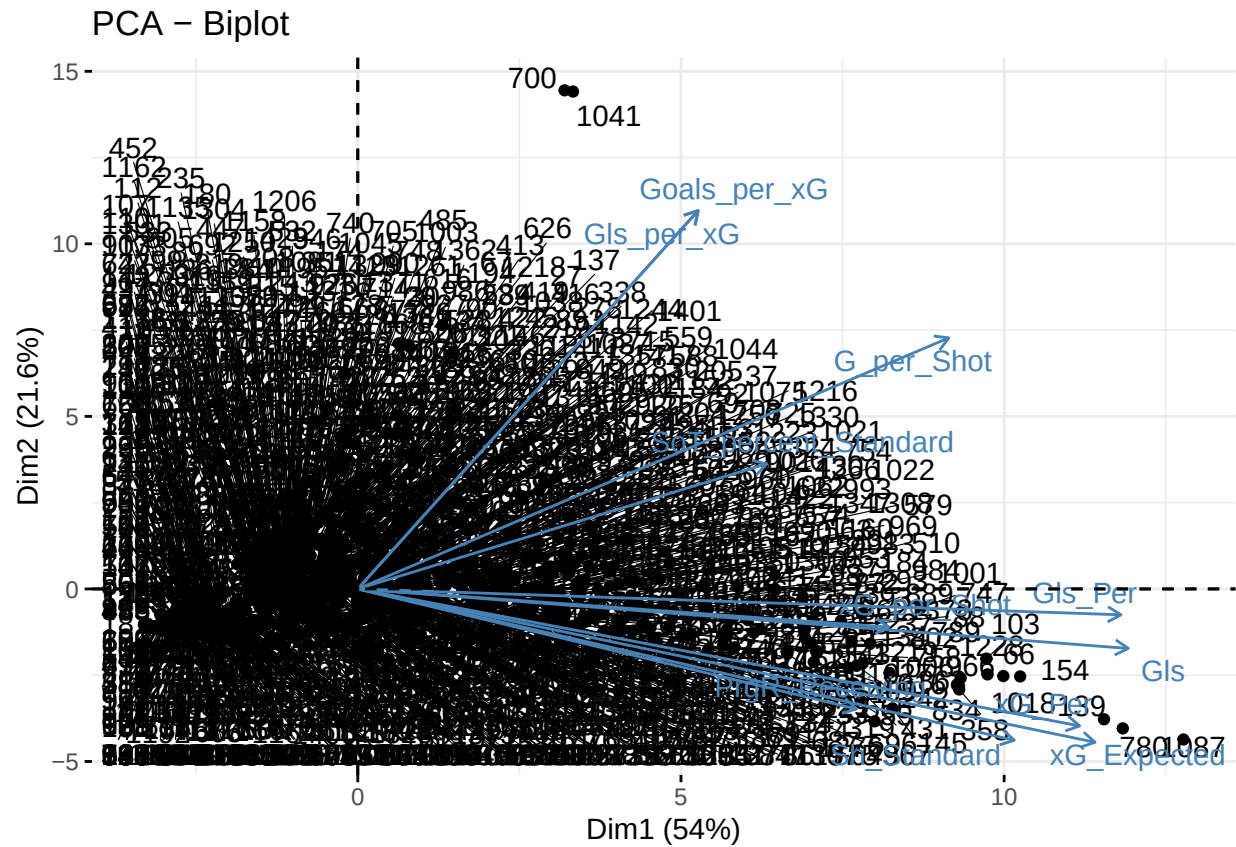
```
# Plot the variance explained
```

```
fviz_eig(outfield_pca)
```

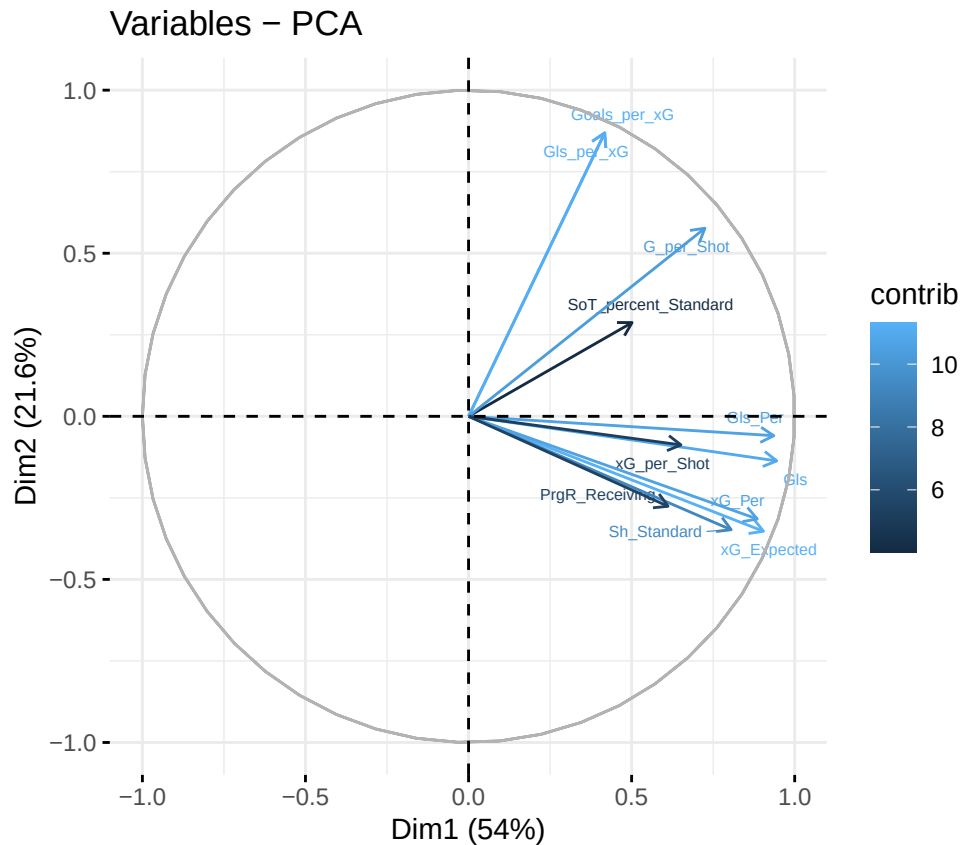
Scree plot



```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```



```
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
  col.var = "contrib",
  labelsize = 2, repel = TRUE)
```



```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

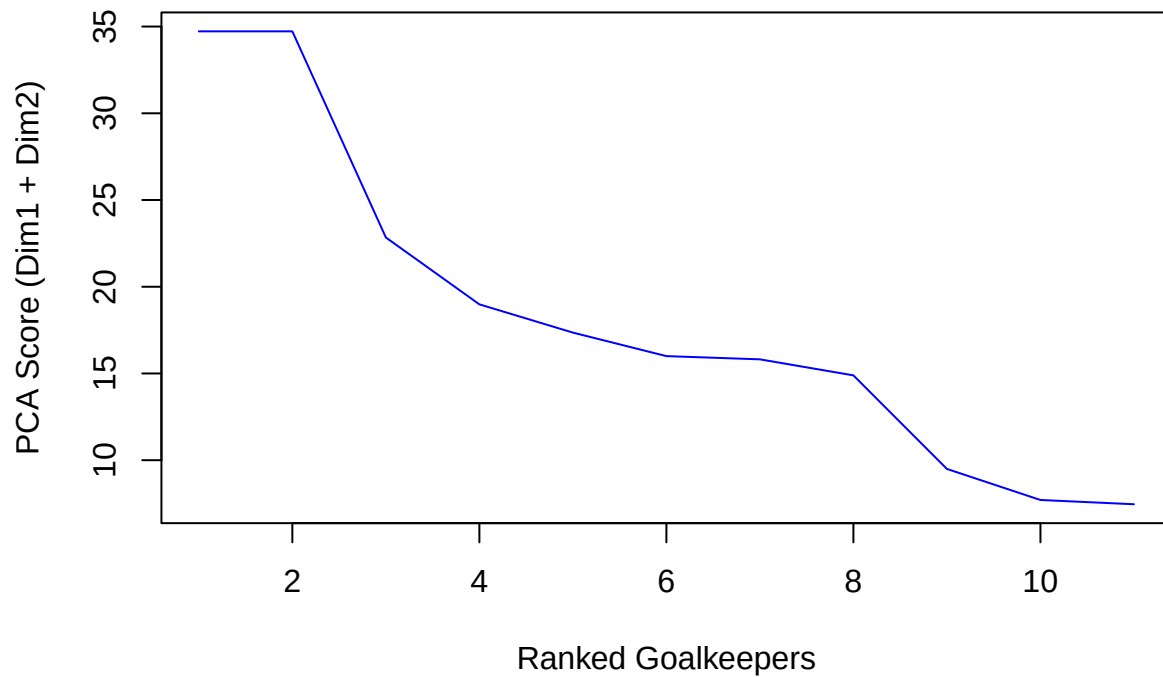
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 20.91283
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##      j1 j2 chi
## ESE  4  8  6
## EDE  4  2 NaN
```

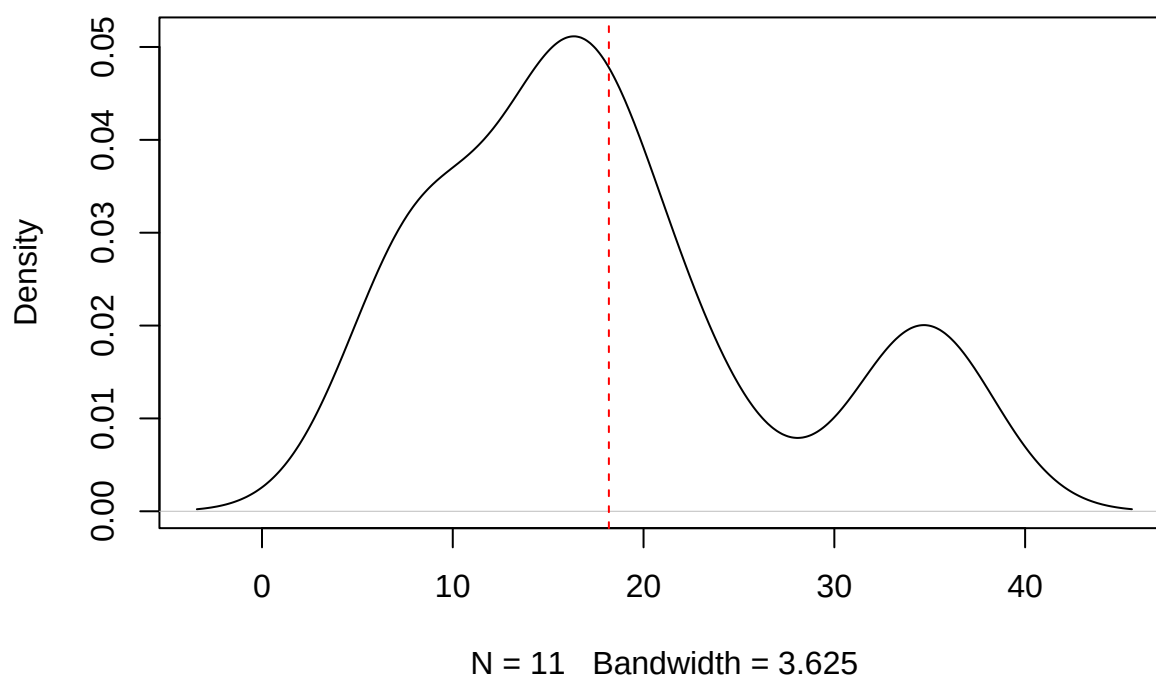
```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 27.6238
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```

Density of PCA Scores



#####

#> GOALSCORER RATING

#Verify if the goals are per 90 or per shot - think they might be per 90

```
outfield_DPR_OCR_gsr <- outfield_gsr_data %>% filter(Min_Playing.Time > 900) %>% mutate(GoalScorer_Rtg =
  (ifelse(xG_Expected > 0, Gls / xG_Expected, 0) * variable_contrib["Gls_per_xG", "Weight"]) +
  (SoT_percent_Standard * variable_contrib["SoT_percent_Standard", "Weight"]) +
  (ifelse(Sh_Standard > 0, Gls / Sh_Standard, 0) * variable_contrib["G_per_Shot", "Weight"]) +
  (xG_per_Shot * variable_contrib["xG_per_Shot", "Weight"]) +
  (PrgR_Receiving * variable_contrib["PrgR_Receiving", "Weight"])
) / Min_Playing.Time) %>% arrange(desc(GoalScorer_Rtg))
```

```
outfield_gsr_awards <- outfield_DPR_OCR_gsr %>% filter(Min_Playing.Time >= 2160)
outfield_GSR <- outfield_DPR_OCR_gsr
```

#Correlation work between GA and GKR

```
cor(outfield_GSR$GoalScorer_Rtg, outfield_GSR$Gls_Per, use = "complete.obs", method = "spearman")
```

Sanity check GSR with xG

```
## [1] 0.6632833
```



```
cor(outfield_GSR$GoalScorer_Rtg, outfield_GSR$xG_Per, use = "complete.obs", method = "spearman")
```

```
## [1] 0.6721444
```

```
summary_model_GSR <- lm(Gls_Per ~ GoalScorer_Rtg, data = outfield_GSR)
summary(summary_model_GSR)$r.squared
```

```
## [1] 0.3816956
```

Reassigning a few table names

```
outfield_OCR <- outfield_DPR_OCR
```

Overall Player Ratings

```
outfield_DPR_filtered <- outfield_DPR %>% select(Squad, Comp, Player, Pos, Poss.Adj.DPR, Min_Playing.Time)
outfield_OCR_filtered <- outfield_OCR %>% select(Squad, Comp, Player, Pos, Poss.adj.OCR, Min_Playing.Time)
outfield_GSR_filtered <- outfield_GSR %>% select(Squad, Comp, Player, Pos, GoalScorer_Rtg, Min_Playing.Time)

player_master <- full_join(outfield_DPR_filtered, outfield_OCR_filtered, by = "Player") %>% full_join(outfield_GSR_filtered, by = "Player")
```

Take the four ratings dataframes and join them before doing clustering on them

```
## Warning in full_join(outfield_DPR_filtered, outfield_OCR_filtered, by = "Player"): Detected an unexpected many-to-many relationship
## i Row 166 of 'x' matches multiple rows in 'y'.
## i Row 1213 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship = "many-to-many"' to silence this warning.
```

```
## Warning in full_join(., outfield_GSR_filtered, by = "Player"): Detected an unexpected many-to-many relationship
## i Row 166 of 'x' matches multiple rows in 'y'.
## i Row 549 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship = "many-to-many"' to silence this warning.
```

```
player_master <- player_master %>%
  mutate(
    DPR_std = (Poss.Adj.DPR - min(Poss.Adj.DPR, na.rm=TRUE)) / (max(Poss.Adj.DPR, na.rm=TRUE) - min(Poss.Adj.DPR, na.rm=TRUE)),
    OCR_std = (Poss.adj.OCR - min(Poss.adj.OCR, na.rm=TRUE)) / (max(Poss.adj.OCR, na.rm=TRUE) - min(Poss.adj.OCR, na.rm=TRUE)),
    GSR_std = (GoalScorer_Rtg - min(GoalScorer_Rtg, na.rm=TRUE)) / (max(GoalScorer_Rtg, na.rm=TRUE) - min(GoalScorer_Rtg, na.rm=TRUE))
  )
```

```
player_master <- player_master %>% mutate(
  Squad= coalesce(Squad, Squad.x, Squad.y),
  Comp= coalesce(Comp, Comp.x, Comp.y),
  Pos = coalesce(Pos.x, Pos.y, Pos),
  Min_Playing_Time = coalesce(Min_Playing.Time.x, Min_Playing.Time.y, Min_Playing.Time))
```

```
player_master_final <- player_master %>% select(Player, Squad, Comp, Pos, Poss.Adj.DPR, Poss.adj.OCR, GSR_std)
```

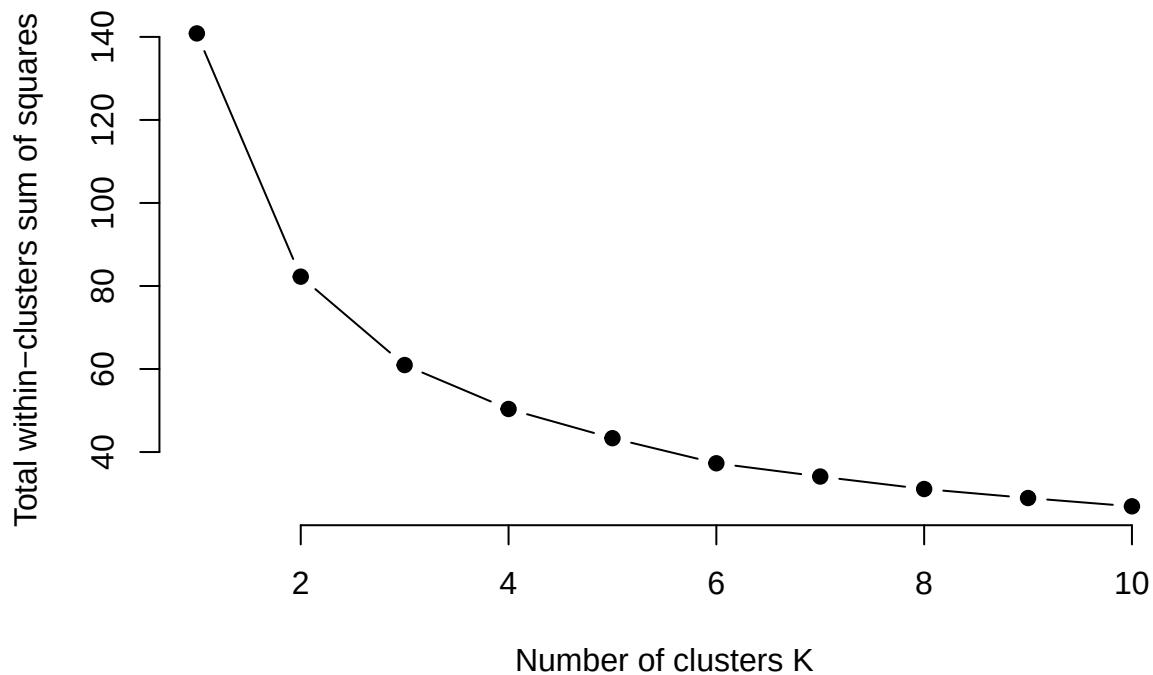
Cluster the players to look for any natural clusterings then run PCA on each cluster

```
player_clustering <- player_master_final %>%
  select(DPR_std, OCR_std, GSR_std) %>%
  mutate(across(everything(), ~ ifelse(is.na(.), 0, .)))

wss <- function(k) {
  kmeans(player_clustering, k, nstart = 25)$tot.withinss
}

k.values <- 1:10
wss_values <- map_dbl(k.values, wss)

plot(k.values, wss_values, type = "b",
     pch = 19, frame = FALSE,
     xlab = "Number of clusters K",
     ylab = "Total within-clusters sum of squares")
```



```
library(cluster)

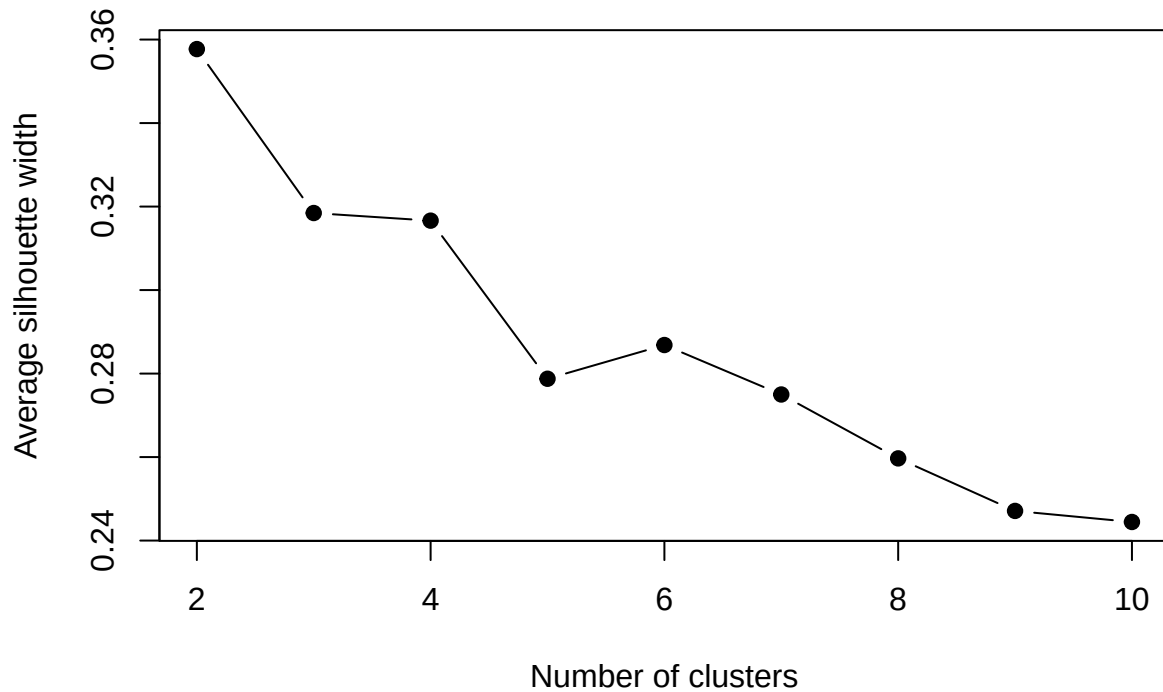
sil_width <- function(k){
  pam(player_clustering, k)$silinfo$avg.width
}
```

```

k.values <- 2:10
sil_values <- map_dbl(k.values, sil_width)

plot(k.values, sil_values, type = "b", pch = 19,
     xlab = "Number of clusters",
     ylab = "Average silhouette width")

```



```

set.seed(234)
kmeans_result <- kmeans(player_clustering, centers = 2, nstart = 25)

player_master_final$Cluster <- kmeans_result$cluster

```

```

run_pca_weights <- function(data_numeric){
  # Perform PCA
  pca <- PCA(data_numeric, graph = FALSE)

  # Extract eigenvalues (variance explained by each component)
  eig <- pca$eig

  # Extract contributions of variables to PC1 and PC2
  var_contrib <- data.frame(pca$var$contrib)

  # Sum contribution of Dim.1 and Dim.2 and scale them to sum to 1
  var_contrib <- var_contrib %>%
    mutate(Total_Contrib = Dim.1 + Dim.2,

```

```

    Weight = Total_Contrib / sum(Total_Contrib))

  # Return PCA object, weights, and eigenvalues
  return(list(pca = pca, weights = var_contrib, eig = eig))
}

player_clustering_final <- player_clustering
player_clustering_final$Player <- player_master_final$Player
player_clustering_final$Squad <- player_master_final$Squad
player_clustering_final$Comp <- player_master_final$Comp
player_clustering_final$Pos <- player_master_final$Pos
player_clustering_final$Cluster <- player_master_final$Cluster
player_clustering_final$Min_Playing_Time <- player_master_final$Min_Playing_Time

clustered_pca_results <- player_clustering_final %>%
  group_split(Cluster) %>%
  map(~run_pca_weights(.x %>% select(DPR_std, OCR_std, GSR_std)))

cluster_weights <- map(clustered_pca_results, ~.x$weights %>% select(Weight) %>% rownames_to_column("Me
names(cluster_weights) <- paste0("Cluster_", 1:length(cluster_weights))

player_clusters <- player_clustering_final %>% group_split(Cluster)

cluster_ratings <- map2(
  player_clusters,
  cluster_weights,
  ~ .x %>% mutate(
    Overall_Rating =
      DPR_std * .y$Weight[.y$Metric == "DPR_std"] +
      OCR_std * .y$Weight[.y$Metric == "OCR_std"] +
      GSR_std * .y$Weight[.y$Metric == "GSR_std"]
  )
)

player_master_final <- bind_rows(cluster_ratings)
player_master_final <- player_master_final %>% arrange(desc(Overall_Rating))

pl_ratings_final <- player_master_final %>% filter(Comp == "Premier League")
laliga_ratings_final <- player_master_final %>% filter(Comp == "La Liga")
ligue1_ratings_final <- player_master_final %>% filter(Comp == "Ligue 1")
bundesliga_ratings_final <- player_master_final %>% filter(Comp == "Bundesliga")
seriea_ratings_final <- player_master_final %>% filter(Comp == "Serie A")

```

Finally decide the awards and the team of the year based on minutes - 2260 minutes played (24 games)

```

player_master_awards <- player_master_final %>% filter(Min_Playing_Time >= 2260)
pl_ratings_awards <- pl_ratings_final %>% filter(Min_Playing_Time >= 2260)
laliga_ratings_awards <- laliga_ratings_final %>% filter(Min_Playing_Time >= 2260)
ligue1_ratings_awards <- ligue1_ratings_final %>% filter(Min_Playing_Time >= 2260)

```

```
seriea_ratings_awards <- seriea_ratings_final %>% filter(Min_Playing_Time >= 2260)
bundesliga_ratings_awards <- bundesliga_ratings_final %>% filter(Min_Playing_Time >= 2260)
```

Overall Team Ratings

```
team_ratings_weighted <- player_master_final %>%
  group_by(Squad, Comp) %>%
  summarise(
    Weighted_Team_Rating = sum(Overall_Rating * Min_Playing_Time, na.rm = TRUE) / sum(Min_Playing_Time,
    Total_Minutes = sum(Min_Playing_Time, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Weighted_Team_Rating))

pl_team_ratings <- team_ratings_weighted %>% filter(Comp == "Premier League")
laliga_team_ratings <- team_ratings_weighted %>% filter(Comp == "La Liga")
ligue1_team_ratings <- team_ratings_weighted %>% filter(Comp == "Ligue 1")
bundesliga_team_ratings <- team_ratings_weighted %>% filter(Comp == "Bundesliga")
seriea_team_ratings <- team_ratings_weighted %>% filter(Comp == "Serie A")
```

No Keeper Ratings

```
pl_ratings_table <- pl_team_ratings %>% select(Squad, Weighted_Team_Rating)
pl_table <- prem_league_table %>% select(Squad) %>% rownames_to_column()
colnames(pl_table)[1] <- "Position"
pl_xg_table <- prem_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

pl_combined_table <- pl_ratings_table %>%
  left_join(pl_table, by = c("Squad")) %>%
  left_join(pl_xg_table, by = c("Squad"))

pl_combined_table$Position <- as.numeric(pl_combined_table$Position)

cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$Position, method = "spearman")
```

Premier League

```
## [1] -0.762406
```

```
cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$Pts, method = "spearman")
```

```
## [1] 0.7700419
```

```
cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$xGD, method = "spearman")
```

```
## [1] 0.7860098
```

```

ggplot(pl_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(pl_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

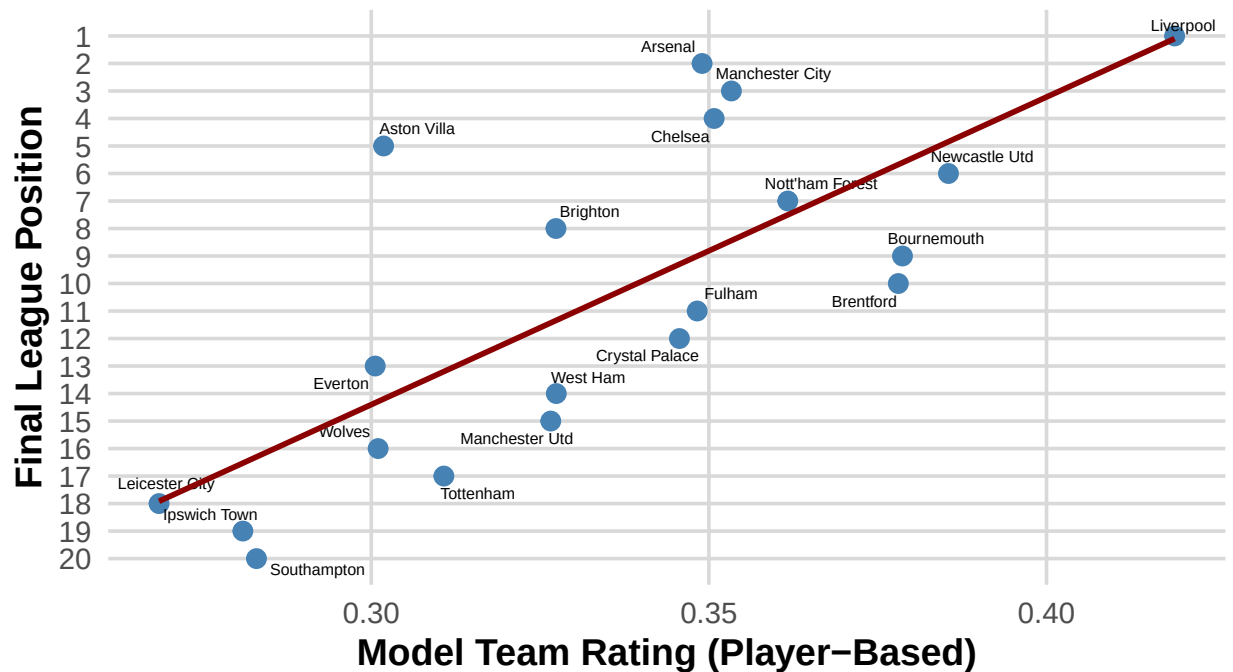
```

## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?

```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



```
laliga_ratings_table <- laliga_team_ratings %>% select(Squad, Weighted_Team_Rating)
laliga_table <- laliga_league_table %>% select(Squad) %>% rownames_to_column()
colnames(laliga_table)[1] <- "Position"
laliga_xg_table <- laliga_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

laliga_combined_table <- laliga_ratings_table %>%
  left_join(laliga_table, by = c("Squad")) %>%
  left_join(laliga_xg_table, by = c("Squad"))

laliga_combined_table$Position <- as.numeric(laliga_combined_table$Position)

cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$Position, method = "spearman")
```

La Liga

```
## [1] -0.7233083
```

```
cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$Pts, method = "spearman")
```

```
## [1] 0.7488722
```

```
cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$xGD, method = "spearman")
```

```
## [1] 0.7714286
```

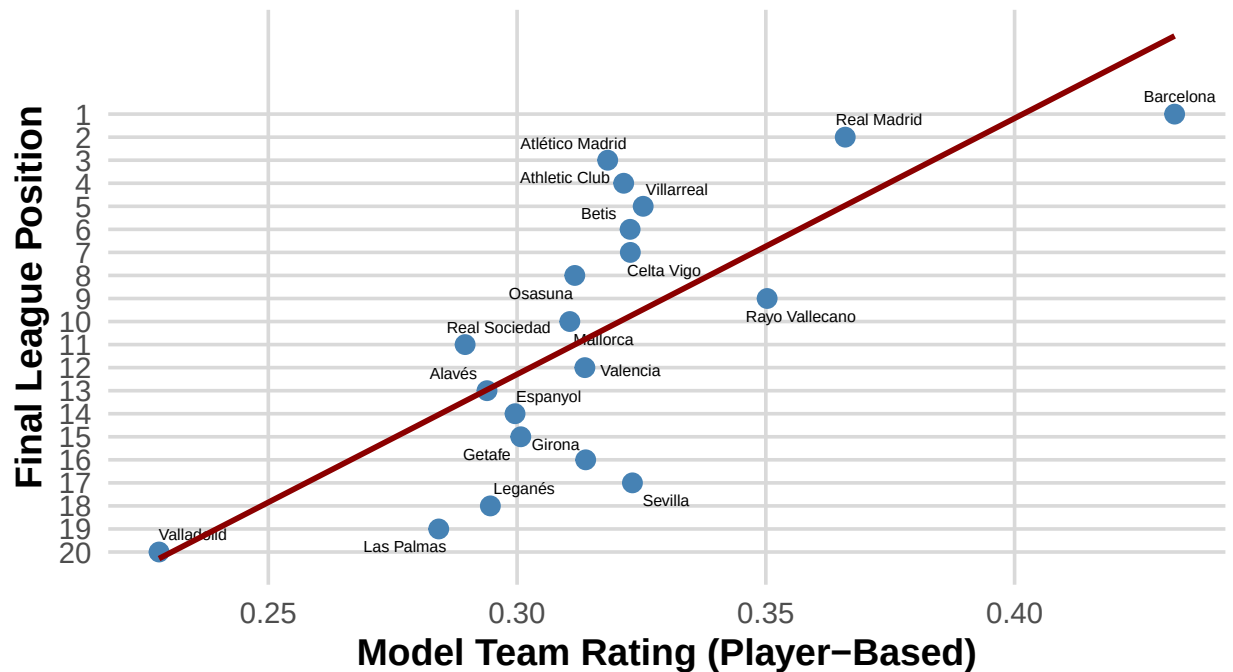
```
ggplot(laliga_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +  
  geom_point(size = 3, color = "steelblue") +  
  geom_text_repel(size = 2, max.overlaps = 20) +  
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +  
  scale_y_reverse(breaks = 1:max(laliga_combined_table$Position, na.rm = TRUE)) +  
  labs(  
    title = "Team Model Rating vs Actual League Position",  
    subtitle = "Higher team rating should correlate with higher league finish",  
    x = "Model Team Rating (Player-Based)",  
    y = "Final League Position",  
    caption = "Line: Linear regression fit | Labels: Squads"  
  ) +  
  theme_minimal(base_size = 14) +  
  theme(  
    plot.title = element_text(face = "bold", size = 18),  
    plot.subtitle = element_text(size = 14, color = "gray40"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.major = element_line(color = "gray85"),  
    panel.grid.minor = element_blank()  
  )
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.  
## i This can happen when ggplot fails to infer the correct grouping structure in  
##   the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
##   variable into a factor?
```


Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



Ligue 1

```

ligue1_ratings_table <- ligue1_team_ratings %>% select(Squad, Weighted_Team_Rating)
ligue1_table <- ligue1_league_table %>% select(Squad) %>% rownames_to_column()
colnames(ligue1_table)[1] <- "Position"
ligue1_xg_table <- ligue1_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

ligue1_combined_table <- ligue1_ratings_table %>%
  left_join(ligue1_table, by = c("Squad")) %>%
  left_join(ligue1_xg_table, by = c("Squad"))

ligue1_combined_table$Position <- as.numeric(ligue1_combined_table$Position)

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$Position, method = "spearman")

## [1] -0.8348813

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$Pts, method = "spearman")

## [1] 0.8304051

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$xGD, method = "spearman")

## [1] 0.7729618

```

```

ggplot(ligue1_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(ligue1_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

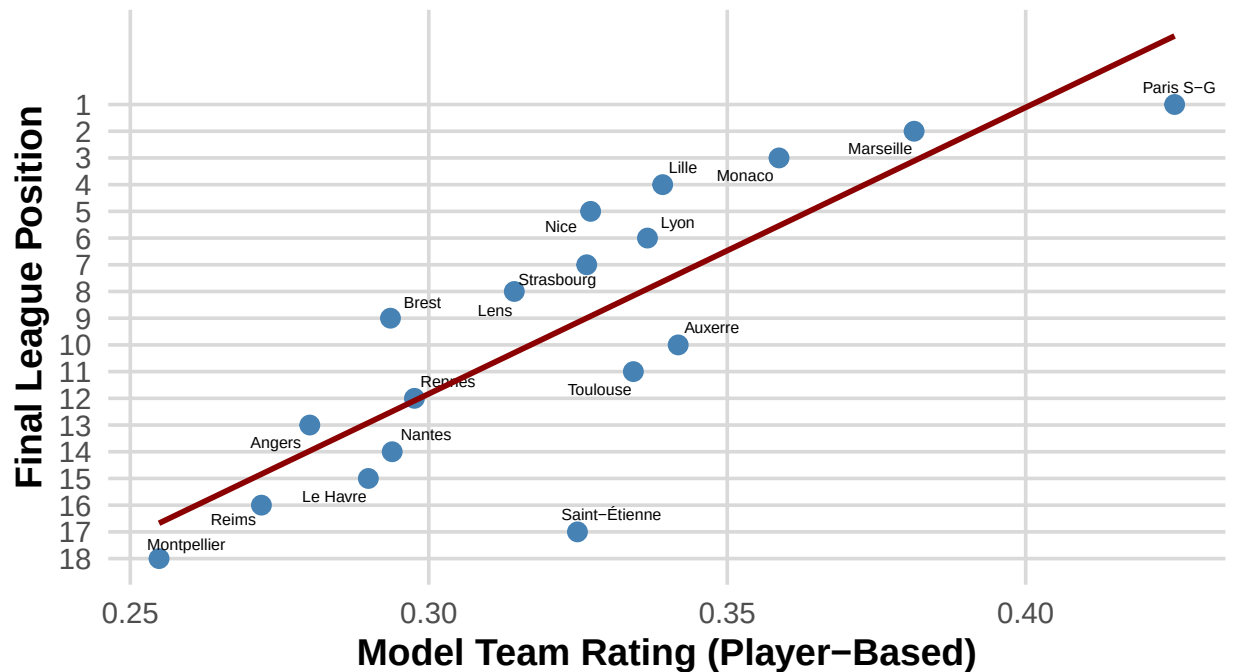
```

## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?

```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



```
bundesliga_ratings_table <- bundesliga_team_ratings %>% select(Squad, Weighted_Team_Rating)
bundesliga_table <- bundesliga_league_table %>% select(Squad) %>% rownames_to_column()
colnames(bundesliga_table)[1] <- "Position"
bundesliga_xg_table <- bundesliga_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

bundesliga_combined_table <- bundesliga_ratings_table %>%
  left_join(bundesliga_table, by = c("Squad")) %>%
  left_join(bundesliga_xg_table, by = c("Squad"))

bundesliga_combined_table$Position <- as.numeric(bundesliga_combined_table$Position)

cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$Position, method = "spearman")
```

Bundesliga

```
## [1] -0.8224974
```

```
cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$Pts, method = "spearman")
```

```
## [1] 0.8221321
```

```
cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$xGD, method = "spearman")
```

```
## [1] 0.8410733
```

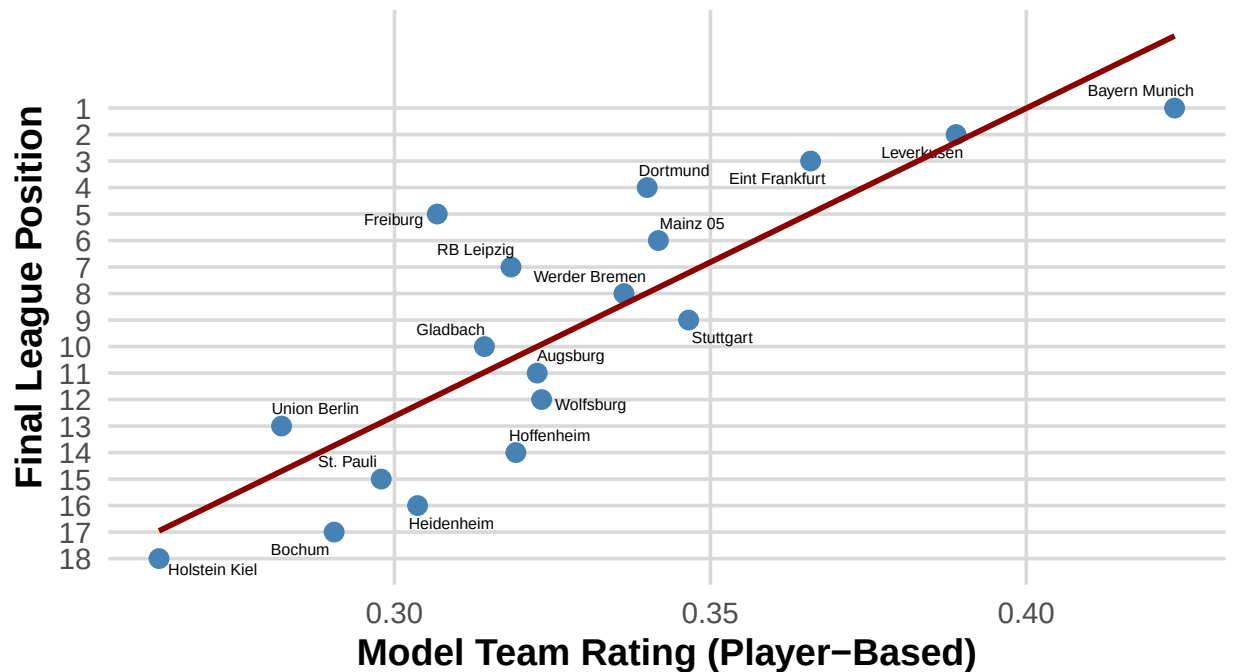
```
ggplot(bundesliga_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +  
  geom_point(size = 3, color = "steelblue") +  
  geom_text_repel(size = 2, max.overlaps = 20) +  
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +  
  scale_y_reverse(breaks = 1:max(bundesliga_combined_table$Position, na.rm = TRUE)) +  
  labs(  
    title = "Team Model Rating vs Actual League Position",  
    subtitle = "Higher team rating should correlate with higher league finish",  
    x = "Model Team Rating (Player-Based)",  
    y = "Final League Position",  
    caption = "Line: Linear regression fit | Labels: Squads"  
  ) +  
  theme_minimal(base_size = 14) +  
  theme(  
    plot.title = element_text(face = "bold", size = 18),  
    plot.subtitle = element_text(size = 14, color = "gray40"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.major = element_line(color = "gray85"),  
    panel.grid.minor = element_blank()  
  )
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.  
## i This can happen when ggplot fails to infer the correct grouping structure in  
##   the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
##   variable into a factor?
```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



```
seriea_ratings_table <- seriea_team_ratings %>% select(Squad, Weighted_Team_Rating)
seriea_table <- seriea_league_table %>% select(Squad) %>% rownames_to_column()
colnames(seriea_table)[1] <- "Position"
seriea_xg_table <- seriea_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

seriea_combined_table <- seriea_ratings_table %>%
  left_join(seriea_table, by = c("Squad")) %>%
  left_join(seriea_xg_table, by = c("Squad"))

seriea_combined_table$Position <- as.numeric(seriea_combined_table$Position)

cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$Position, method = "spearman")
```

Serie A

```
## [1] -0.8045113
```

```
cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$Pts, method = "spearman")
```

```
## [1] 0.811442
```

```
cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$xGD, method = "spearman")
```

```
## [1] 0.8556391
```

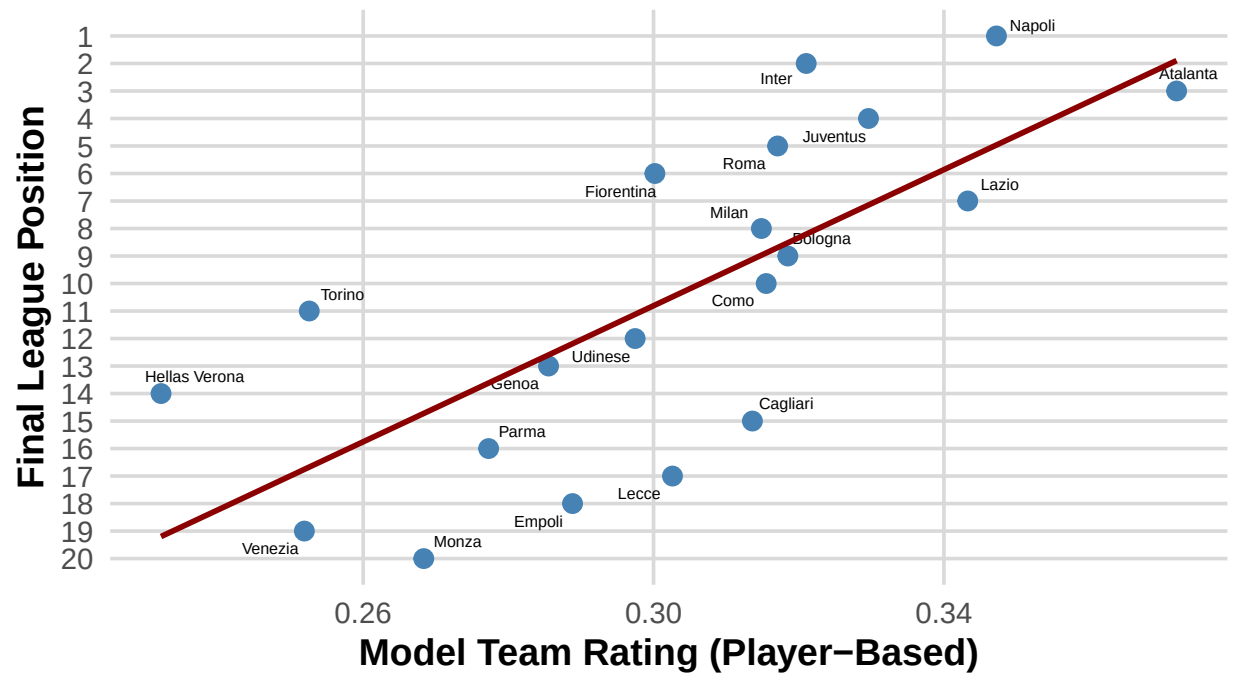
```
ggplot(seriea_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +  
  geom_point(size = 3, color = "steelblue") +  
  geom_text_repel(size = 2, max.overlaps = 20) +  
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +  
  scale_y_reverse(breaks = 1:max(seriea_combined_table$Position, na.rm = TRUE)) +  
  labs(  
    title = "Team Model Rating vs Actual League Position",  
    subtitle = "Higher team rating should correlate with higher league finish",  
    x = "Model Team Rating (Player-Based)",  
    y = "Final League Position",  
    caption = "Line: Linear regression fit | Labels: Squads"  
  ) +  
  theme_minimal(base_size = 14) +  
  theme(  
    plot.title = element_text(face = "bold", size = 18),  
    plot.subtitle = element_text(size = 14, color = "gray40"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.major = element_line(color = "gray85"),  
    panel.grid.minor = element_blank()  
  )
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.  
## i This can happen when ggplot fails to infer the correct grouping structure in  
##   the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
##   variable into a factor?
```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



Line: Linear regression fit | Labels: Squads